

Notes Link:

<https://bit.ly/oracle730pm>

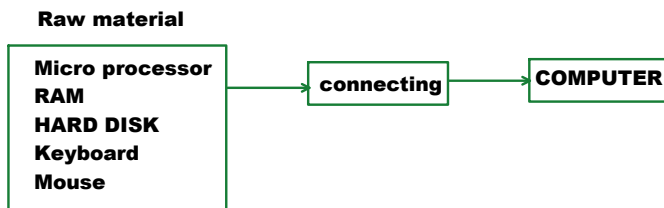
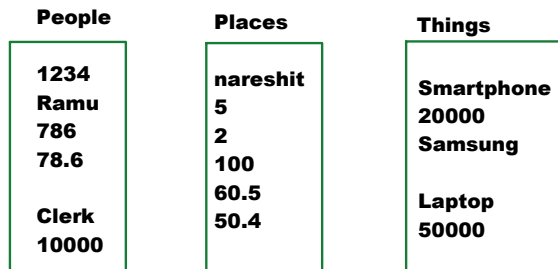
Oracle DataBase 19c installation process

<https://bit.ly/oracleinstall>

Data
Information
Database
DBMS
RDBMS
Metadata

Data:

- Data is a raw collection of facts about people, places, things ...etc.



Data:

- Data is a raw collection of facts about people or places or things..etc.
- It is unprocessed one.
- It is not in meaningful form.
- It cannot be used for data analysis.

Information:

- If we arrange the data in meaningful form then it is called "Information".
- It is processed one.
- It is in meaningful form.
- It can be used for Data Analysis.

Types of Data:

2 types:

- Structured Data
- Unstructured Data

Structured data:

It is made up of with letters, digits & special symbols.

Ex: Ramu 1234 H.NO.1-2-123

Unstructured Data:

Images, audios, videos, documents, animations are comes under unstructured data.

employee

empno	ename	job	sal	ephoto
1234	Kiran	CLERK	5000	10110001001001010

Database:

Bank DB

Customers
Transactions
Branches
Staff

College DB

Student
Marks
Fee
Staff
Library

Amazon DB

Products
Customers
Orders

- Database means, complete details of an organization stored in computer in specific manner.
- Database is a collection of interrelated data in an organized form.

College DB

STUDENT		
sid	sname	scity
1001	Ravi	Hyd

MARKS			
sid	math	phy	che
1001	60	80	70

FEE			
sid	total_fee	paid_fee	balance
1001	15000	10000	5000

Interrelated data

STUDENT			Table / Relation / Entity
sid	sname	scity	
1001	Ravi	Hyd	row / tuple / entity instance / record
1002	Kiran	Delhi	
1003	Ramu	Mumbai	Column / Field / Attribute / Property

Table:
is a collection of rows & columns

Field:
holds individual values

Record:

is a collection of field values

Database contains records & fields

DBMS:

- DBMS => DataBase Management Software / System
- It is a software that is used to create & maintain the database.
- Using this software, we can store, manipulate & retrieve the data of database.
 - manipulate => INSERT / UPDATE [modify] / DELETE
 - retrieve => opening existing data

RDBMS:

- RDBMS => Relational DataBase Management Software / System.
- Relation => Table
- RDBMS is a software that is used to create & maintain the database in the form of tables [relations].

Ex:

ORACLE SQL SERVER My SQL DB2 Postgre SQL

Laptop RDBMS

apple	oracle
dell	sql server
hp	my sql
asus	db2

Metadata:

- It is the data about the data.
 - It describes about the data.
 - Ex:
 - Table name [STUDENT_INFO]
 - column name [sid,sname,scity]
 - Data type
 - Field size => [sid => number(4)]
- Handwritten notes: "Data Type" with an arrow pointing to "number(4)" and "Field Size" with an arrow pointing to "(4)".*

STUDENT_INFO

sid	sname	scity
1234	Ramu	Hyd
1235	Kiran	Mumbai

int
float
char

sid Number(4) => -9999 to 9999

1234 *Handwritten notes: "Data Type" with an arrow pointing to "Number(4)" and "Field Size" with an arrow pointing to "(4)".*
1235
1236

sname Varchar2(10)

sname	Varchar2(10)

Vijay	
Arun	

ORACLE

- ORACLE is a Relational DataBase Management Software [RDBMS]
- Relation => Table
- ORACLE is used to create & maintain database in the form of tables.
- This software allows us to store, manipulate & retrieve the data of database.
- manipulate => INSERT / UPDATE / DELETE
 - Ex:
 - emp joined => INSERT
 - emp got promotion => UPDATE
 - emp resigned => DELETE
- RETRIEVE => opening existing data
 - Ex: check balance online shopping
- ORACLE software is product of ORACLE company.
- ORACLE software 2nd version introduced in 1979.
- ORACLE 21C is latest version => 2021.

ORACLE => RDBMS

RDBMS => Concept => introduced in 1976 => E.F.Codd

Before 1960s => data manually in books

1960s => FMS [File Management System]

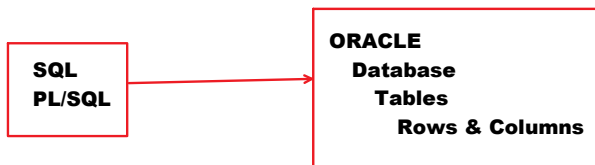
1970s => Hierarchical & Network DBMSs

ORACLE company owner => LARRY ELLISON

1977 => Software Development Laboratories

1979 => Relational Software Inc.

1983 => ORACLE corp.



To deal with ORACLE database, we use 2 languages:

- SQL
- PL/SQL

SQL:

- SQL => Structured Query Language
- SQL is a Query Language that is used to write the queries.
- Query => is a request that is sent to DataBase Server.
- Queries are written to communicate with ORACLE database.
- SQL is a Non-Procedural Language. We will not write any set of statements.
- SQL is a 4GL [4th Generation language]. In 4GLs, we much focus on what to do rather than how to do. These 4GLs provides readymade commands, functions to make our tasks easier.
- SQL provides operators to perform operations.
- SQL provides built-in functions.

C /Java / Python

Programming Languages
programs

Function [in C]:
set of statements

Method [in Java]:
set of statements

Procedure [PL/SQL]:
set of statements

ORACLE SQL provides 5 sub Languages. They are:

DDL Data Definition Language	CREATE ALTER TRUNCATE
---------------------------------	-----------------------------

	DROP RENAME FLASHBACK [Oracle 10g] PURGE [Oracle 10g]
DRL / DQL Data Retrieval Language Data Query Language	SELECT
DML Data Manipulation Language	INSERT UPDATE DELETE INSERT ALL [Oracle 9i] MERGE [Oracle 9i]
DCL / ACL Data Control Language Accessing Control Language	GRANT REVOKE
TCL Transaction Control Language	COMMIT ROLLBACK SAVEPOINT

DDL :

- DDL stands for Data Definition Language.
- It deals with metadata.

SQL provides following DDL commands:

- Create
- Alter
- Drop
- Truncate
- Rename
- Flashback [Oracle 10g]
- Purge [Oracle 10g]

CREATE:

- It is used to create the DataBase Objects like tables, views, indexes ...etc.

Syntax to CREATE the table:

```
CREATE TABLE <table_name>
(
  <field_name> <data_type> [constraint <con_name> <con_type>,
  <field_name> <data_type> constraint <con_name> <con_type>,
  <field_name> <data_type> constraint <con_name> <con_type>,
  .
.]
);
```

DB Objects

- Table
- View
- Index
- Materialized view
- Sequence
- Synonym
- Procedure
- Function
- Package

Data Types in SQL:

- Data Type tells which type of data a column has to accept.

employee

empno	ename	job	sal	deptno
-------	-------	-----	-----	--------

ORACLE SQL provides following data types:

Number Related	Number(p)	Integer Rel Int
	Int	
	Integer	

Number Related ✖	Number(p) ✓ Int Integer	Integer Related
	✖ ✖ ✖ Number(p,s) ✓ Float binary_float binary_double Real	
Character Related	Char(n) Varchar2(n) LONG CLOB nChar(n) nVarchar2(n) nCLOB n => national	
Date & Time Related	Date Timestamp (Oracle 9i)	
Binary Related	BFILE BLOB	

Number Related Data Types:

Number(p):

- It is used to hold integers.
- Integer means, number without decimal places.
- p => precision => max no of digits
- precision valid range => 1 to 38

Ex:

max marks: 100

0 to 100

maths_marks Number(3) 100 => 3 digits

78

100

999

1000 --ERROR

-999 TO 999

1000 --ERROR

123456 -- ERROR

Addhar_num Number(12)

Mobile_num Number(10)

cust_id Number(6)

123456

123457

123458

age Number(2)

18

23

21

25

Number(38) = INT = INTEGER

```
CREATE TABLE t3
(
  f1 number(38),
  f2 int,
  f3 integer
);
```

Number(p,s):

- p => precision => max number of digits
- s => scale => max no of decimal places
- It is used to hold floating point values.

Ex:

3 subject

avrg **Number(5,2)**

 -999.99 TO 999.99
 78.23
 83.45
 100.00
 999.99
 1000 --ERROR

math => 100
 phy => 100
 che => 100

average => 100.00

5,2

sal **Number(8,2)** **max sal => 100000.00**

999999.99

height **Number(2,1)**

5.3
 5.2 -9.9 TO 9.9
 5.8
 5.9
 6.0

Character Related data types:

Char(n):

- n => max no of chars
- used to hold a group of chars [string values].
- Fixed Length Data Type.
- max size: 2000 bytes.
- default size: 1

Varchar2(n):

- n => max no of chars
- used to hold a group of chars [string values].
- Variable Length data type.

- max size: 4000 bytes.
- default size: no default size

f1 Char(10)	f2 Varchar2(10)
ramu6spaces	ramu
ramesh4spaces	ramesh
sai7spaces	sai

Handwritten notes:

- For f1 Char(10): ramu (6 spaces), ramesh (4 spaces), sai (7 spaces). Total length is 10 for each row.
- For f2 Varchar2(10): ramu (4), ramesh (6), sai (3). Total length is 10 for each row.
- Labels: "fixed length" for f1, "variable length" for f2.

create table t5(f1 char); --default size 1

create table t6(f1 varchar2); --no default size
--ERROR

Vehicle_num Char(10)	ename Varchar2(10)
TS09AA1234	ramu
	kiran
	naresh
	sai

PAN_CARD_NUM Char(10)	pname varchar2(15)
ABCDE 12345	laptop
	keyboard
	hard disk

Gender Char(1)
M
F

StateCOde Char(2)
AP
TS
MH
WB

CountryCode Char(3)
IND
AUS
PAK
USA

LONG:

- used to hold string values.
- max size: 2 GB.
- It is used to hold large amounts of chars.
- In one table, we create one column as LONG type.
- We cannot use built-in functions on LONG type.

Ex:

Customer_Feedback
Experience summary
Cust_Complaints
Product_features

CLOB:

- CLOB => Character Large Object
- Max size: 4 GB
- it is used to hold string values.
- it is used to hold large amounts of chars.

Ex:**Customer_Feedback****Experience summary****Cust_Complaints****Product_features**

Char(n) Varchar2(n) LONG CLOB	ASCII CODE char data types Single Byte Char data types supports to ENGLISH language chars only.
nChar(n) nVarchar2(n) nCLOB	UNI CODE char data types Multi Byte char data types supports to ENGLISH + other language chars also

n => national character set data types

In C =>

char x; // 1 Byte = 8 bits => ASCII
0 to 255
255 => 1111 1111 => 8 bits
256 => 9 bits
English chars only

In Java:

char x; // 2 bytes => UNI code
0 to 65535

65535 => 1111 1111 1111 1111 => 16 bits
65536 => 17 bits

Java => English + other lang chars

Date & Time related data types:**Date:**

- It is used to hold date values.
- It can hold date, month, year, hours, minutes & seconds.
- Default date format: dd-mon-rr
- Ex: 23-DEC-19
- default time: 12:00:00 AM [mid night]
- Fixed length data type.
- size: 7 bytes

Ex:**DOBirth****DOJoining****DORetirement****ord_date****del_date****yy****2019****2078****RR**

	2022	2070
00-49	19 => 2019 25 => 2025	19 => 2119 25 => 2125
50-99	78 => 1978 85 => 1985	78 => 2078 85 => 2085

21st century

2000 to 2049
2050 to 2099

Timestamp:

- used to hold date & time values.
- It can hold date, month, year, hours, minutes, seconds & fractional seconds.
- Fixed length data type
- size: 11 bytes
- It is extension of date data type.

Ex

date_of_joining

.....

bill_date_time

.....

Date	Timestamp
7 bytes	11 bytes
It cannot hold fractional seconds	It can hold fractional seconds
it cannot display time value by default. to see it, we use "to_Char()" function	it displays time values by default.

Binary related data types:

BFILE:

- used to maintain multimedia objects like images, audios, videos, animations, documents.
- It maintains path of multimedia object.
- External Large Object.
- It is not secured one

Database

emp		
empno	ename	ephoto {BFILE}
1234	ramu	d:\employees\ramu.jpg

d:
employees folder
ramu.jpg



BLOB:

- used to maintain multimedia objects like images, audios, videos, animations, documents.
- multimedia object will be stored inside of table.
- Internal Large Object.

emp		
empno	ename	ephoto {BLOB}
1234	ramu	1253AB8745CDF123EF

d:
employees folder
ramu.jpg



Database

It is secured one.

Log In as DBA:

username: system

password: nareshit

[at the time of installation u have given this password]

CREATING USER:

Syntax:

```
CREATE USER <user_name>
IDENTIFIED BY <password>
DEFAULT TABLESPACE <tablespace_name>
QUOTA <size> ON <tablespace_name>;
```

TABLESPACE	is a file. in this oracle data will be stored
------------	---

There are 2 types of users:

- Local User
- Common User

Creating Common user:

Common user name must be prefixed with c##.

Ex:

```
CREATE USER c##batch4pm
IDENTIFIED BY nareshit
DEFAULT TABLESPACE users
QUOTA unlimited ON users;
```

```
GRANT connect,resource TO c##batch4pm;
```

Examples on creating tables:

Ex-1:

STUDENT

sid	sname	m1
-----	-------	----

Syntax of CREATE command:

```
CREATE TABLE <table_name>
(
  <field_name> <data_type> [constraint <con_name> <con_type>],
  <field_name> <data_type> constraint <con_name> <con_type>,
  <field_name> <data_type> constraint <con_name> <con_type>,
  .
.]
);
```

CREATE TABLE student

(

STUDENT

CREATE TABLE student

```
(  
  sid Number(4),  
  sname Varchar2(10),  
  m1 Number(3)  
);
```

STUDENT

sid	sname	m1
-----	-------	----

ORACLE => RDBMS => create & maintain the database

SQL
PL/SQL

SQL => 5 sub languages

DDL
DRL
DML
DCL
TCL

DDL:

CREATE
ALTER
DROP
TRUNCATE
RENAME
FLASHBACK
PURGE

CREATE:

```
CREATE TABLE <name>  
(  
  <field_name> <data_type> [constraint <con_name> <con_type>],  
  <field_name> <data_type> constraint <con_name> <con_type>,  
  <field_name> <data_type> constraint <con_name> <con_type>  
  .  
.]  
);
```

STUDENT1

sid	sname	M1
1001	Ramu	70
1002	Kiran	50

100

sid	Number(4)
sname	Varchar2(10)
M1	Number(3)

CREATE TABLE student1

```
(  
  sid Number(4),  
  sname Varchar2(10),  
  m1 number(3)  
);
```

Output: Table created.

To see table structure:

Syntax:

DESC <table_name>;

DESC student1;

Output:

**sid Number(4)
sname Varchar2(10)
m1 number(3)**

To see all tables list created by user:

user_tables:

It maintains all tables information

SELECT table_name FROM user_tables;

Output:

Table_name

**T1
STUDENT
STUDENT1**

Inserting Records:

Syntax of INSERT command:

**INSERT INTO <table_name>[(<column_list>)]
VALUES(<value_list>);**

Inserting single record:

INSERT INTO student1 VALUES(1001,'Ramu',70);

Output: 1 row created.

INSERT INTO student1 VALUES(1002,'Kiran',50);

Output: 1 row created.

COMMIT; --save

Inserting multiple records using parameters:

"parameter" concept is used to read value at run time.

&sid => parameter

INSERT INTO student1 VALUES(&sid,'&sname',&m1);

**enter value for sid: 1003
enter value for sname: Sai
enter value for m1: 80**

/

**enter value for sid: 1004
enter value for sname: Sravan
enter value for m1: 66**

/

**enter value for sid: 1005
enter value for sname: Vijay
enter value for m1: 77**

To see table data:

SELECT * FROM student1;

Inserting limited column values:

sid	sname	m1
1006	Prasad	

```
INSERT INTO student1(sid,sname)
VALUES(1006,'Prasad');
```

Inserting limited column values by
changing order:

1007	Sravan	
------	--------	--

```
INSERT INTO student1(sname,sid)
VALUES('Sravan',1007);
```

Example-2:

Employee

empno	ename	job	sal	doj
5001	A	CLERK	5000	25-dec-2021
5002	B	MANAGER	8000	20-AUG-2019

empno	Number(4)	100000.00
ename	Varchar2(10)	Number(8,2)
job	Varchar2(10)	
sal	Number(8,2)	
doj	Date	

```
CREATE TABLE employee
(
  empno Number(4),
  ename Varchar2(10),
  job Varchar2(10),
  sal Number(8,2),
  DOJ Date
);
```

Output: Table created.

Inserting record:

```
INSERT INTO employee
VALUES(5001,'A','CLERK',6000,'25-DEC-2021');
```

```
INSERT INTO employee
VALUES(5002,'B','MANAGER',8000,'17-aug-2019');
```

```
insert into employee
values(&empno,&ename,&job,&sal,&doj);
Enter value for empno: 5003
Enter value for ename: C
Enter value for job: CLERK
Enter value for sal: 15000
Enter value for doj: 13-oct-2018
old 1: insert into employee
values(&empno,&ename,&job,&sal,&doj)
new 1: insert into employee
values(5003,'C','CLERK',15000,'13-oct-2018')
```

1 row created.

SQL>

SQL> /

Enter value for empno: 1004

Enter value for ename: D

Enter value for job: salesman

Enter value for sal: 4000

Enter value for doj: 16-sep-2021

old 1: insert into employee

values(&empno,&ename,&job,&sal,&doj)

new 1: insert into employee

values(1004,'D','salesman',4000,'16-sep-2021')

1 row created.

employee

empno	ename	job	sal	doj
5005	E		9000	

INSERT INTO employee(empno,ename,sal)
VALUES(5005,'E',9000);

Constraint:

- Constraint => restrict / limit / control

✓ Check (maths_marks >= 0 AND maths_marks <= 100) max marks: 100

maths_marks
78
59
625

0 TO 100

55 ✓

79 ✓

429 → ERROR

625 → ERROR

Gender

M	✓
F	✓
F	✓
M	✓
Z	→ ERROR → Invalid data

Check (Gender = 'M' OR Gender = 'F')

Constraint:

- Constraint => Restrict / Limit / Control
- Constraint is a rule that is applied on a column.
- Constraint restricts the user from entering invalid data.
- It is used to implement Data Integrity.
- Data Integrity means, maintaining accurate & quality data.

ORACLE SQL provides following Constraints:

- Primary Key
 - Not Null
 - Unique
 - Default
 - Check
 - References [Foreign Key]
- duplicate
null

Primary Key:

- should not accept duplicate values
- should not accept null values

Ex: *primary key*

<i>empno</i>	ename	job	sal
1001	Raju	Clerk	6000
1002	Kiran	Clerk	5000
1003	Raju	Manager	10000
1004	Sai	Clerk	5000
1001	Arun	Manager	15000
	Sravan	Manager	12000

duplicate
X

Null
X

Not Null:

- It does not accept null value.
- It accepts duplicate value.

Ex:

Employee

empno	ename	sal
1001	Ramu	5000
1002	Kiran	7000
1003	Ramu	6000
1004		10000

Not Null

Duplicate ✓
Null X
It demands for ename.

Unique:

- It does not accept duplicate values.
- It accepts null values.

Ex:

STUDENT

AdmnNo	Sname	Scity
1234	Ramu	Hyd
1235	Kiran	Mumbai
	Vijay	Pune
1234	Arun	Delhi

Unique

Null ✓
Duplicate X

Customer

cid	cname	gmail_id	mobile_num
1001	A	abc@gmail.com	90123 45678
1002	B		90123 45678
1003	C	qwe@gmail.com	90123 45678

Unique
Unique
Null ✓
Null ✓
Duplicate X

1003 C qwe@gmail.com 0123 45678
X duplicate

Constraint	Duplicate	Null
Primary Key	X	X
Not Null	✓	X
Unique	X	✓

PK = Not Null + Unique

Default:

- It is used to apply default value to a column.
- When for all records value is same then apply default value for that column.

STUDENT

sid	sname	college_name	college_city	Fee
1	Ramu	nareshit	hyd	15000
2	Sai	nareshit	hyd	15000
3	Vijay	nareshit	hyd	15000
				6000

default 'nareshit'
default 'hyd'
default 15000

Check:

- It is used to apply our own conditions on column.

sid	sname	m1
1	A	67
2	B	78
3	C	554

max marks: 100

0 TO 100

Check (m1) >= 0
AND
m1 <= 100
554 >= 0 = T
554 <= 100 = F
All T

References [Foreign Key]:

- Foreign Key refers to Primary Key values of another table.
- IN Foreign Key column, another table's Primary key values only accepted. Other than those values will not be accepted.

Ex:

COURSE

PK

cid	cname
10	Java
20	SQL Server
30	Python
40	C#.Net

STUDENT

sid	sname	cid
1	A	30
2	B	10
3	C	40
4	D	80

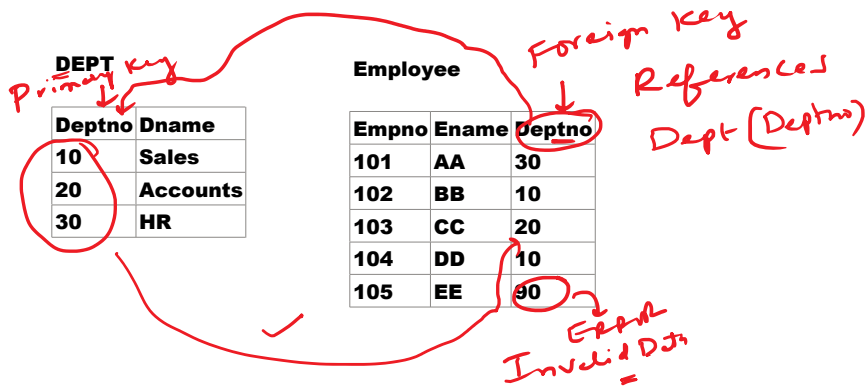
Foreign Key
References
Course (cid)

→ ERROR
Invalid data

DEPT

Employee

Foreign Key
References



Constraint => Rule applied on column

PK	not accept duplicate not accept null
Unique	not accept duplicate accepts null
Not Null	not accept null accepts duplicate
Check	to apply our own conditions
Default	to apply default value to column
References {FK}	Foreign Key refers to PK values of another table.

CREATE:

```

CREATE TABLE <name>
(
  <field_name> <data_type> [constraint <con_name> <con_type>,
  <field_name> <data_type> constraint <con_name> <con_type>,
  <field_name> <data_type> constraint <con_name> <con_type>
  .
]
);

```

Examples on Constraints:

Handwritten notes: "PK" pointing to sid; "Not Null" pointing to M1; "Check" pointing to M1; "Max marks: 100" and "0 to 100" circled; "dup x" and "null x" pointing to duplicate and null values in the table.

STUDENT2

sid	sname	M1
1001	A	67
1002	B	50
1001		
	C	88
1003		90
1004	A	50

sid	Number(4)	PK
sname	Varchar2(10)	Not Null
M1	Number(3)	Check

```

CREATE TABLE student2
(
  sid Number(4) primary key,
  sname Varchar2(10) not null,

```

m1 Number(3) Check(m1>=0 AND m1<=100)
);
Output: Table created.

INSERT INTO student2 VALUES(1,'A',70);
Output: 1 row created.

INSERT INTO student2 VALUES(2,'B',50);
Output: 1 row created.

INSERT INTO student2 VALUES(1,'C',60);
Output: ERROR: unique constraint violated

INSERT INTO student2 VALUES(null,'C',60);
Output: Error: cannot insert null into student2.sid

INSERT INTO student2 VALUES(3,'A',65);
Output: 1 row created.

INSERT INTO student2 VALUES(4,null,60);
Output:
Error: cannot insert null into student2.sname

INSERT INTO student2 VALUES(5,'E',789);
Output: ERROR: check constraint violated.

Example-2:

STUDENT3

sid	sname	cname	ccity	Fee
1001	A	nareshit	Hyd	15000
1002	B	nareshit	Hyd	15000
1003	A	nareshit	Hyd	15000
1004	D	nareshit	Hyd	15000
1001	E	nareshit	Hyd	15000

Handwritten notes: Unique (pointing to sid), Default 'nareshit' (pointing to cname), Default 'Hyd' (pointing to ccity), Default 15000 (pointing to Fee). Red circles and arrows highlight the duplicate '1001' in the sid column and the default values.

sid	Number(4)	Unique
sname	Varchar2(10)	
cname	Varchar2(10)	DEFAULT
ccity	Varchar2(10)	Default
Fee	Number(7,2)	Default

```
CREATE TABLE student3
(
  sid Number(4) Unique,
  sname Varchar2(10),
  cname Varchar2(10) Default 'nareshit',
  ccity Varchar2(10) Default 'Hyd',
  Fee Number(7,2) Default 15000
);
```

Output: Table created.

INSERT INTO student3 VALUES(1,'A');
Output: ERROR: not enough values

INSERT INTO student3(sid,sname) VALUES(1,'A');
Output: 1 row created.

INSERT INTO student3(sid,sname) VALUES(2,'B');
Output: 1 row created.

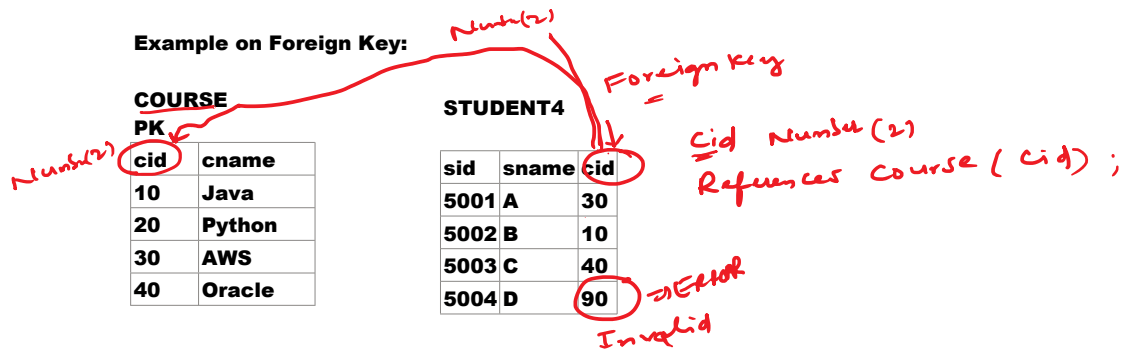
INSERT INTO student3(sid,sname) VALUES(1,'C');
Output: ERROR: unique constraint violated

INSERT INTO student3(sid,sname) VALUES(null,'D');
Output: 1 row created.

INSERT INTO student3(sid,sname) VALUES(5,'E');
Output: 1 row created.

SELECT * FROM student3;

Example on Foreign Key:



Course

cid	Number(2)	PK
cname	Varchar2(10)	

STUDENT4

sid	Number(4)	
sname	Varchar2(10)	
cid	Number(2)	Foreign Key Course cid

CREATE TABLE course

(
cid Number(2) Primary Key,
cname Varchar2(10)
);

Output: Table created.

CREATE TABLE student4

(
sid Number(4),
sname Varchar2(10),
cid Number(2) REFERENCES course(cid)
);

NOTE:

- To set foreign key, PK column and FK column data type & field size should be matched
- Column name need not to be same.

INSERT INTO course VALUES(10,'JAVA');
INSERT INTO course VALUES(20,'PYTHON');
INSERT INTO course VALUES(30,'AWS');
INSERT INTO course VALUES(40,'ORACLE');
COMMIT;

SELECT * FROM course;

INSERT INTO student4(1001,'A',30);
output: 1 row created.

INSERT INTO student4(1002,'B',10);
output: 1 row created.

INSERT INTO student4(1002,'B',90);
output: error: integrity constraint violated

Naming Constraints:

- We can give names to the constraints.
- "constraint" keyword is used to define name for

constraint.

- When we define a constraint, it's better to give constraint name. If we don't define constraint name ORACLE defines a name implicitly.
- ORACLE defined name will be prefixed with "sys_c"
- "sys_c" & 6 digit random number will be taken as constraint name.
- To disable or enable or drop the constraints name is required.

Creating Table with constraint types & names:

student11

sid	sname	m1
-----	-------	----

sid	Number(4)	PK	c1
sname	Varchar2(10)	Not Null	c2
m1	Number(3)	Check	c3

```
CREATE TABLE student11
(
  sid number(4) constraint c1 primary key,
  sname varchar2(10) constraint c2 not null,
  m1 number(3) constraint c3 check(m1>=0 and m1<=100)
);
```

Output: Table created.

user_constraints:

It maintains all constraints information.

DESC user_constraints;

```
SELECT constraint_name, constraint_type,
table_name FROM user_constraints;
```

```
SELECT constraint_name, constraint_type,
table_name FROM user_constraints
WHERE table_name='STUDENT11';
```

SQL:

Queries

5 sub languages

DDL:

CREATE

ALTER

DROP

TRUNCATE

RENAME

FLASHBACK

PURGE

ALTER:

- ALTER => Change

STUDENT

sid|sname|scity|GENDER

after some days => add a column GENDER

- Using ALTER command we can change structure of the table.

- Using ALTER command we can:

- add the columns → ADD
- rename the columns → RENAME COLUMN
- drop the columns → DROP
- modify the field sizes → MODIFY
- modify the data types → MODIFY
- add the constraints → ADD CONSTRAINT
- rename the constraints → RENAME
- disable constraints → DISABLE
- enable the constraints → ENABLE
- drop the constraints → DROP

Syntax of ALTER command:

```
ALTER TABLE <table_name> [ADD(<field_definitions>)]
                        [RENAME COLUMN <old_column_name>] TO <new_col_name>]
                        [DROP COLUMN <column_name>]
                        [DROP(<colun_list>)]
                        [MODIFY(<new_field_definitions>)]
                        [ADD CONSTRAINT <constraint_name> <con_type(<column>)]
                        [RENAME CONSTRAINT <old_name> TO <new_name>]
                        [DISABLE CONSTRAINT <constraint_name>]
                        [ENABLE CONSTRAINT <constraint_name>]
                        [DROP CONSTRAINT <constraint_name>;]
```

Example on ALTER command:

STUDENT

sid|sname

CREATE TABLE student

```
(
sid number(4),
sname varchar2(10)
);
```

Output: Table created.

Adding a column [m1 column]:

ALTER TABLE student ADD m1 number(3);

Output: Table Altered.

Desc student;

sid
sname
m1

Adding Multiple Columns [m2, m3 columns]:

**ALTER TABLE student ADD(m2 number(3),
m3 number(3));**

Changing column name [change m3 col name as maths]:

**ALTER TABLE student
RENAME COLUMN m3 TO maths;**

Dropping one column [maths column]:

```
ALTER TABLE student
DROP COLUMN maths;
```

--we can drop one column only using **DROP COLUMN**

Dropping multiple columns [m1 & m2]:

```
ALTER TABLE student
DROP(m1,m2);
```

Modifying data type [sid => number(4) => char(10)]:

```
ALTER TABLE student
MODIFY sid char(10);
```

Modifying the field size [sname => varchar2(10) => 10 to 20] :

```
ALTER TABLE student
MODIFY sname varchar2(20);
```

Can we decrease field size?

Yes. But, we can decrease up to max string length in column data.

sname → varchar2(20)
↓

Ramu — 4
Sravan — 6
Kiran — 5
naresh — 6
sai — 7
↓
varchar2(6)
↓
up to 6 only we can decrease
=

Adding Constraint:

[Add PK to sid]

```
ALTER TABLE student
ADD CONSTRAINT c5 Primary Key(sid);
```

Disabling Constraint:

```
ALTER TABLE student
DISABLE CONSTRAINT c5;
-- temporarily PK will not work
```

Enabling Constraint:

```
ALTER TABLE student
ENABLE CONSTRAINT c5;
```

Renaming Constraint:

```
ALTER TABLE student
RENAME CONSTRAINT c5 TO abc;
```

Dropping Constraint:

**ALTER TABLE student
DROP CONSTRAINT abc;**

DDL Commands:

**CREATE
ALTER
DROP
FLASHBACK
PURGE
TRUNCATE
RENAME**

**DROP
FLASHBACK
PURGE**

DROP:

- used to drop [delete] the table.
- When we drop the table, it goes to **RECYCLEBIN**.

Syntax:

DROP TABLE <table_name> [PURGE];

Ex:

DROP TABLE employee;

-- employee table will be placed in recyclebin

To see recyclebin [dropped tables list]:

show recyclebin

employee

FLASHBACK:

- used to restore the dropped table.

Syntax:

FLASHBACK TABLE <table_name> TO BEFORE DROP;

Ex:

FLASHBACK TABLE employee TO BEFORE DROP;

PURGE:

- is used to delete dropped table from recyclebin.

Syntax:

PURGE TABLE <table_name>;

Ex:

PURGE TABLE employee;

Dropping table permanent:

DROP TABLE employee;
PURGE TABLE employee;

OR

DROP TABLE employee PURGE;

Emptying recyclebin:

PURGE recyclebin;

DROP	drop the table. it goes to recyclebin
FLASHBACK	restore the dropped table
PURGE	to delete table from recyclebin

TRUNCATE:

- it is used to delete all records from the table.
- it deletes complete information from the table.

Syntax:

TRUNCATE TABLE <table_name>;

Ex:

TRUNCATE TABLE student;

- all records will be deleted.
- we cannot recollect truncated data.

student

=> TABLE

sid	sname
1	A
2	B
3	C

Structure

+

Data

TRUNCATE	it deletes table data. it will not delete table structure.
DROP	it deletes entire table. It means, table data + table structure will be deleted

RENAME:

- used to rename the database objects like tables.

Syntax:

RENAME <old_name> TO <new_name>;

Ex:

RENAME student3 TO std;

DDL

CREATE	to create the tables
---------------	----------------------

ALTER	to change structure of table
DROP	to drop the table
FLASHBACK	to restore from recyclebin
PURGE	to delete from recyclebin
TRUNCATE	to delete all records
RENAME	to change the table name

DRL / DQL:

- DRL => Data Retrieval Language
- DQL => Data Query Language
- Retrieval => opening existing data
- Query => is a request that is sent to the database
- DRL delas with data retrievals.

ORACLE SQL provides only one DRL command. i.e:
SELECT

SELECT:

- used to retrieve the data from database.

Syntax:

default

```

SELECT [DISTINCT/ALL] <column_list / *>
FROM <table_list>
[WHERE <condition>]
[GROUP BY <grouping_Column_list>]
[HAVING <group_Condition>]
[ORDER BY <column_list> ASC/DESC];

```

ENGLISH
SENTENCES
WORDS

SQL
QUERIES
CLAUSES

CLAUSE => part of the query

Using **SELECT** command we can retrieve:

- single record
- a set of records
- all records
- limited columns
- all columns

Displaying single record:

Display the emp record whose name is SMITH:

```

SELECT * FROM emp
WHERE ename='smith';

```

no rows selected

NOTE:

- SQL is not case sensitive. But, string comparison is case sensitive.

'smith' = 'SMITH' => FALSE
no rows selected

'SMITH' = 'SMITH' => TRUE
displays smith record

column list
Tell name

```

SELECT empno,ename,sal FROM emp
WHERE ename='SMITH';

```

```

SELECT empno,ename,sal FROM emp
WHERE ename='SMITH';

```

-- displays SMITH record

Handwritten notes: "Take name" points to ename, "Condition" points to WHERE.

SELECT	to specify the column list
FROM	to specify the table name
WHERE	to write the condition on rows

Execution Order:

```

FROM
WHERE
SELECT

```

Display the emp record whose empno is 7499:

```

SELECT * FROM emp
WHERE empno=7499;

```

Display a set of records:

Display all managers records:

```

SELECT empno,ename,job
FROM emp
WHERE job='MANAGER';

```

Display all records:

```

SELECT * FROM emp;

```

Display limited columns:

emp

empno	ename	job	sal	mgr	comm	hiredate	deptno
-------	-------	-----	-----	-----	------	----------	--------

ename	job	sal
-------	-----	-----

```

SELECT ename,job,sal FROM emp;

```

Display all columns:

```

SELECT * FROM emp WHERE deptno=10;

```

Operators in SQL:

- Operator:
- Operator is a symbol that is used to perform operations like arithmetic operations or logical operations.

Oracle SQL provides following operators:

Arithmetic	+ - * /				
Relational (or) Comparison	<	>	<=	>=	=
					equals
				!= (or) <> (or) ^=	not equals

IN C/Java:

```

+   -   *   /   5%2
5/2 = 2

```

IN SQL:

Arithmetic	+ - * /
Relational (or) Comparison	< > <= >= = != (or) <> (or) ^= equals not equals
Logical	AND OR NOT
SET	UNION UNION ALL INTERSECT MINUS
Special	IN NOT IN BETWEEN AND NOT BETWEEN AND IS NULL IS NOT NULL LIKE NOT LIKE
Concatenation	
Other	Any All Exists Not Exists PIVOT UNPOVOT

+ - * / 5%2
5/2 = 2

IN SQL:
+ - * /
5/2 = 2.5

mod() => function
mod(5,2) => remainder 1

Arithmetic Operators:
are used to perform arithmetic operations.

+	Addition
-	Subtraction
*	Multiplication
/	Division

Examples on Arithmetic Operators:

Calculate 100+200:

SELECT 100+200 FROM dual;

100+200

300

SELECT 100+200 as total FROM dual;

TOTAL

300

SELECT 100+200 total FROM dual;

TOTAL

300

SELECT 100+200 as "total" FROM dual;

total

300

SELECT 100+200 as total marks FROM dual;
ERROR:

SELECT 100+200 as total_marks FROM dual;

TOTAL_MARKS

300

SELECT 100+200 as "total marks" FROM dual;

total marks

300

Column Alias:

- Column Alias is another name or alternative name of a column.
- To give column alias we use "AS" keyword.
- Using "AS" keyword is optional.
- to maintain case or to give alias in multiple words specify alias name in double quotes.

Dual:

- "DUAL" is a built-in table [readymade table].
- DUAL table created in "sys" schema [user].
- It has 1 column & 1 row
- column name is: DUMMY Varchar2(1) => X [data].

SYS

DUAL
DUMMY
X

SELECT 100+200 FROM emp;

emp => 14 rows

300
300
300
.
.
300 => 14 times

SELECT 100+200 FROM dual;

300 => dual => 1row

- If our requirement is getting one value as the result then use DUAL.
- to work with non-database values, we can use DUAL.

Calculate annual salary:

emp

empno	ename	sal
1001	A	5000
1002	B	8000
1003	C	6000

**SELECT empno,ename,sal,
sal*12 as "Annual Salary"
FROM emp;**

Calculate TA, HRA, TAX & GROSS salaries:

employee

empno	ename	sal
1001	A	6000
1002	B	9000
1003	C	4000

TA => 10% on sal
HRA => 20% on sal
TAX => 5% on sal
GROSS => sal+TA+HRA-TAX

```

SELECT empno,ename,sal,
sal*0.1 as TA,
sal*0.2 as HRA,
sal*0.05 as TAX,
sal+sal*0.1+sal*0.2-sal*0.05 as GROSS
FROM emp;

```

Assignment:

sid	sname	M1	M2	M3
1	A	60	90	80
2	B	65	78	34

calculate total & average marks

Relational Operators / Comparison Operators:
are used to compare 2 values

ORACLE SQL provides following Relational Operators:

>	greater than
<	less than
>=	greater than or equals to
<=	less than or equals to
=	equals to
!= / <> / ^=	no equals to

Display the emp records whose salary is more than 2500:

```

SELECT empno,ename,sal
FROM emp
WHERE sal>2500;

```

Display the emp records whose salary is 3000 or more:

```

SELECT empno,ename,sal
FROM emp
WHERE sal>=3000;

```

Display the emp records whose salary is less than 2000:

```

SELECT empno,ename,sal
FROM emp
WHERE sal<2000;

```

Display th emp records whose salary is 1500 or less:

```

SELECT empno,ename,sal
FROM emp
WHERE sal<=1500;

```

Display the emp records who are working in 30

dept:

```
SELECT empno,ename,deptno
FROM emp
WHERE deptno=30;
```

Display all clerks records:

```
SELECT empno,ename,job
FROM emp
WHERE job='CLERK';
```

Display all emps records except clerks:

```
SELECT empno,ename,job
FROM emp
WHERE job!='CLERK';
```

Numbers:

1
2 > 1
3
4
5
.
.
10

Date

1-jan-2022

2-jan-2022

3-jan-2022

4-jan-2022

.

.

31-dec-2022

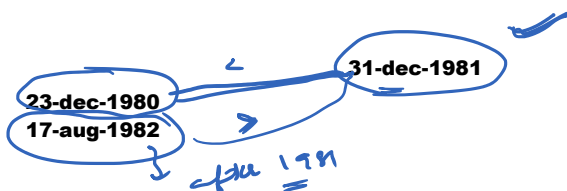
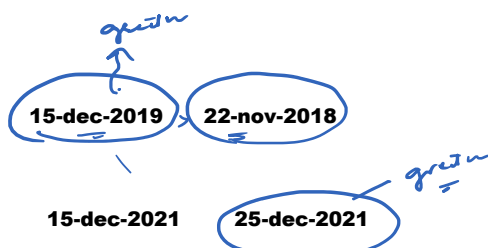
lowest
=

> 1-jan-2022

>

>

→ highest
date



Display the emp records who joined after 1981:

```
SELECT empno,ename,hiredate
FROM emp
WHERE hiredate>'31-dec-1981';
```

Display the emp records who joined before

1981:

```
SELECT empno,ename,hiredate
FROM emp
WHERE hiredate<'1-jan-1981';
```

LOGICAL OPERATORS:

are used to perform logical operations.

ORACLE SQL provides following Logical Operators:

- AND
- OR
- NOT

AND, OR operators are used to separate multiple conditions.

AND	All conditions should be satisfied
OR	At least one condition should be satisfied

Truth Table:

condition1	condition2	con1 AND con2	con1 OR con2
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

STUDENT

Max marks: 100

Min marks: 40 for pass in each

sid	sname	M1	M2	M3
1001	A	60	90	80
1002	B	50	30	70

Display the students records whose result is PASS:

```
SELECT * FROM student
WHERE m1>=40 AND m2>=40 AND m3>=40;
```

1001	A	60	90	80
------	---	----	----	----

Display the students records whose result is FAIL:

```
SELECT * FROM student
WHERE M1<40 OR M2<40 OR M3<40;
```

1002	B	50	30	70
------	---	----	----	----

Display all managers records whose salary is more than 2500:

```
SELECT empno,ename,job,sal
FROM emp
WHERE job='MANAGER' AND sal>2500;
```

Display all managers records whose salary is more than 2500 and hiredate must be after april 1981:

```
SELECT empno,ename,job,sal,hiredate
FROM emp
WHERE job='MANAGER' AND sal>2500
AND hiredate>'30-apr-1981';
```

Display all managers and clerks records:

```

SELECT empno,ename,job
FROM emp
WHERE job='MANAGER' OR job='CLERK';

```

AND	used to perform logical AND operations [ALL T]
OR	used to perform logical OR operations [Min 1 T]

NOT:
is used to perform logical NOT operations.

condn	Not(condn)
T	Not(T) => F
F	Not(F) => T

Display all managers records:

```

SELECT empno,ename,job FROM emp
WHERE job='MANAGER';

```

Display all emp records except managers:

```

SELECT empno,ename,job FROM emp
WHERE NOT(job='MANAGER');

```

Special Operators:
Special operators are also comparison operators.

IN:

Syntax:

```

<column_name> IN(<value_list>)

```

- used to compare a column value with a list of values.
- It avoids of writing multiple equality conditions using **OR**

Display the emp records whose empnos are 7499,7521,7900:

```

SELECT empno,ename,sal
FROM emp
WHERE empno=7499 OR empno=7521 OR empno=7900;

```

```

empno=7499 OR empno=7521 OR empno=7900

```

[OR]

```

empno IN(7499,7521,7900)

```

```

SELECT empno,ename,sal FROM emp
WHERE empno IN(7499,7521,7900);

```

Display the emp records whose names are:
SMITH, BLAKE AND WARD:

```
SELECT empno,ename,sal
FROM emp
WHERE ename IN('SMITH','BLAKE','WARD');
```

Display all managers and clerks records:

```
SELECT empno,ename,job
FROM emp
WHERE job IN('MANAGER','CLERK');
```

Display all emp records except clerks:

```
SELECT empno,ename,job
FROM emp
WHERE job NOT IN('CLERK');
```

Display all emp records who are working in
10 & 30 depts:

```
SELECT empno,ename,deptno
FROM emp
WHERE deptno IN(10,30);
```

Display all emp records who are not working
in deptno 30:

```
SELECT empno,ename,deptno
FROM emp
WHERE deptno NOT IN(30);
```

BETWEEN AND:

Syntax:

<column_name> BETWEEN <lower> AND <upper>

- It is used to compare column value with a range of values.

Examples:

Display the emp records whose salary is 2000 or greater and 3000 or less:

```
SELECT empno,ename,sal
FROM emp
WHERE sal BETWEEN 2000 AND 3000;
```

```
SELECT empno,ename,sal
FROM emp
WHERE sal BETWEEN 3000 AND 2000;
```

What si the output of query?

- a) **ERROR** b) displays records c) no rows selected
d) none of above

Display the emp records whose sal is <2000 or >3000:

```
SELECT empno,ename,sal
FROM emp
WHERE sal NOT BETWEEN 2000 AND 3000;
```

Display the emp records who joined in 1981:

who joined after 1981	hiredate>'31-dec-1981'
who joined before 1981	hiredate<'1-jan-1981'

```
SELECT empno,ename,hiredate
FROM emp
WHERE hiredate BETWEEN '1-jan-1981' AND '31-dec-1981';
```

Display the emp records who are not joined in 1981:

```
SELECT empno,ename,hiredate
FROM emp
WHERE hiredate NOT BETWEEN '1-jan-1981' AND '31-dec-1981';
```

Display the emp records who joined in 1982,1983 & 1984:

```
SELECT empno,ename,hiredate
FROM emp
WHERE hiredate BETWEEN '1-jan-1982' AND '31-dec-1984';
```

LIKE:

Syntax:

<column_name> LIKE <text_pattern>

- used to compare column value with text pattern.

SQL provides 2 wildcard characters. They are:

%	replaces 0 or any no of chars
_	replaces 1 char

Display the emp records whose name is started with 'S':

```
SELECT empno,ename,sal
FROM emp
WHERE ename LIKE 'S%';
```

Display the emp records whose names are ended with S:

```
SELECT empno,ename,sal
FROM emp
WHERE ename LIKE '%S';
```

IN Windows OS,

to search for all jpg files	*.jpg
to search for all mp3 files	*.mp3
to search for all jpg files which are started with S	s*.jpg
to search for all jpg files in which 2nd char is a	?a*.jpg

Wildcard chars:

*	replaces 0 or any no of chars
?	replaces 1 char

Display the employee records whose name is started with 'A':

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE 'A%';
```

**Display the emp records whose name is ended with
"RD":**

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE '%RD';
```

Display the emp records whose name is having 2nd char as A:

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE '_A%';
```

Display the emp records whose name's 3rd char is 'R':

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE '__R%';
```

Display the emp records whose names are having 'A' char:

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE '%A%';
```

Display the emp records whose names are having 4 chars:

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE '____';
```

Display the emp records whose names are having 6 chars:

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE '_____';
```

Display the emp records who are getting 3 digit salary:

```
SELECT empno,ename,sal  
FROM emp  
WHERE sal LIKE '___';
```

**Display the emp records whose names are not started with
'S':**

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename NOT LIKE 'S%';
```

NULL:

- **NULL means blank or empty**
- **When the value is unknown or unable to insert we give NULL.**

STUDENT

sid	sname	scity
1001	Ravi	Hyd
	Sai	Delhi

NULL

sid	sname	M1
1001	A	50
1002	B	0
1003	C	

NULL

space) $\neq$ NULL
0 $\neq$ NULL

NULL:

- NULL means blank or empty
- When the value is unknown or unable to insert we give NULL.
- Null is not equals to 0.
- Null is not equals to space.
- If NULL is participated in operation then definitely result will be NULL.

Ex:

$100 + \text{NULL} = \text{NULL}$

$10 + 20 * \text{null} = \text{NULL}$

- For Null comparison we cannot use = [equals to] operator. For Null comparison we must use "IS NULL" operator.

NULL value can be inserted in 2 ways:

- using NULL keyword
- by inserting limited column values

Ex:

CUSTOMER

cid	cname	ccity
-----	-------	-------

CREATE TABLE customer

```
(  
  cid number(4),  
  cname varchar2(10),  
  ccity varchar2(10)  
);
```

INSERT INTO customer VALUES(1,'A','Hyd');

1st way:

Inserting NULL using NULL keyword:

INSERT INTO customer VALUES(2,'B',null);

2nd way:

Inserting NULL by inserting limited column values:

INSERT INTO customer(cid,cname)
VALUES(3,'C');

IS NULL:

used to compare column value with NULL.

Syntax:

<column_name> IS NULL;

Display the emp records who are not getting commission:

```
SELECT empno,ename,sal,comm
FROM emp
WHERE comm IS null;
```

Display the emp records who are getting commission:

```
SELECT empno,ename,sal,comm
FROM emp
WHERE comm IS not null;
```

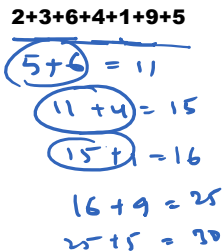
Concatenation Operator:
Symbol: ||

used to combine the 2 strings.

Ex:

```
SELECT 'Raj' || 'Kumar' FROM dual;
```

RajKumar



```
SELECT 'Raj' || ' ' || 'Kumar' FROM dual;
```

Raj Kumar

```
SELECT empno, ename FROM emp;
```

empno	ename
-----	-----

```
SELECT empno || ' ' || ename as "empno_name"
FROM emp;
```

empno_name

7369 SMITH
7499 ALLEN

SQL

DDL	create alter drop flashback purge truncate rename
DRL / DQL	SELECT
TCL	commit rollback

	savepoint
DML	Insert Update Delete Insert All Merge

TCL:

- **TCL => Transaction Control Language**
- **It deals with transactions.**
- **Transaction is a series of actions [commands].**

ORACLE SQL provides following TCL commands:

- **COMMIT [Save]**
- **ROLLBACK [Undo All]**

Ex:

withdraw Transaction

Reads card info => **SELECT**
select withdraw
enter amount => **5000 SELECT**
success withdraw!
UPDATE the balance => **UPDATE**

5000
10000

Transaction must be successfully completed or aborted [cancelled].

COMMIT [Save]:

- **It is used to save the transactions.**
- **When commit command is executed all the changes in ORACLE INSTANCE [RAM] will be applied to ORACLE DATABASE [HARD DISK].**

Syntax:

COMMIT;

Ex:

COMMIT;
-- saves the transaction

NOTE:

ORACLE SERVER = Instance + Database
[RAM] [Hard Disk]

Rollback:

- **is used to cancel the previous actions.**
- **it cancels uncommitted actions.**
- **After COMMIT, we cannot use ROLLBACK.**

Syntax:

ROLLBACK [TO <save_point_name>];

Ex: ROLLBACK;

Example on ROLLBACK & COMMIT:

Test1

f1	f2
1	A
2	B

Note:

All DDL commands are auto-committed.

All DML commands are not auto-committed.

DDL command = DDL command + COMMIT

CREATE TABLE test1

```
(
f1 number(4),
f2 varchar2(10)
);
```

--committed

INSERT INTO test1 VALUES(1,'A');
INSERT INTO test1 VALUES(2,'B');

Select * FROM test1;

1	A
2	B

Rollback;

Select * FROM test1;

no rows selected

INSERT INTO test1 VALUES(1,'A');
INSERT INTO test1 VALUES(2,'B');

Select * FROM test1;

1	A
2	B

COMMIT;

ROLLBACK;

Select * FROM test1;

1	A
2	B

Case-1

CREATE => t1 => CREATE + COMMIT => committed

INSERT
INSERT
INSERT
INSERT

Rollback; — 4 actions cancelled

CREATE t1

INSERT

INSERT

CREATE t2

INSERT

committed
CREATE + COMMIT

```

INSERT
INSERT
CREATE t2 → CREATE + commit ✓
INSERT
INSERT
Rollback; → 2 actions cancelled ✓

```

SavePoint:

- used to set margin for Rollback.

Syntax:

Savepoint <save_point_name>;

Example on Savepoint:

CREATE Test1

```

SavePoint aaa;
INSERT INTO Test1 Values(11,'A');
INSERT INTO Test1 Values(12,'B');
SavePoint bbb;
INSERT INTO Test1 Values(13,'C');
INSERT INTO Test1 Values(14,'D');
SavePoint ccc;
INSERT INTO Test1 Values(15,'E');
INSERT INTO Test1 Values(16,'F');

```

Rollback TO aaa; -- 6 actions will be cancelled
 Rollback TO bbb; --4 actions will be cancelled
 Rollback TO ccc; -- 2 actions will be cancelled
 Rollback; -- 6 actions will be cancelled

TCL commands:

COMMIT [save]	used to save the transaction when commit is executed all changes of oracle instance will be applied to oracle database.
ROLLBACK	used to cancel previous actions it cancels uncommitted actions
SAVEPOINT	to set margin for rollback we use it

DML:

- DML => Data Manipulation Language
- Manipulation => Insert / Update / Delete
 - emp joined => INSERT
 - emp got promotion => UPDATE
 - emp resigned => DELETE
- It deals with the data.
- All DML commands are not auto-committed.
- After performing DML operation,
 - to save it use COMMIT.
 - to cancel it use ROLLBACK.

ORACLE SQL provides following DML commands:

- INSERT
- UPDATE

- DELETE
- INSERT ALL
- MERGE

UPDATE:

- it is used to modify the data.
- using this command we can update:
 - single value of single record
 - multiple values of single record
 - a set of records
 - all records
 - using parameters

Syntax:

```
UPDATE <table_name>
SET <field_name>=<value>[,<field_name>=<value>,
<field_name>=<value>, .....]
[WHERE <condition>];
```

Updating single value of single record:

```
UPDATE emp SET sal=sal+1000
WHERE empno=7369;
```

Updating multiple values of single record:

```
UPDATE emp SET job='MANAGER', sal=6000
WHERE empno=7369;
```

Updating a set of records:

Increase 10% on sal to all managers:

```
UPDATE emp SET sal=sal+sal*0.1
WHERE job='MANAGER';
```

Updating all records:

Increase 20% on sal to all emps:

```
UPDATE emp SET sal=sal+sal*0.2;
```

Updating records using parameters:

Increase => 10% on sal => empno 7499

Increase => 20% on sal => empno 7900

Increase => 15% on sal => empno 7934

```
UPDATE emp SET sal=sal+sal*&per/100
WHERE empno=&empno;
enter value for per:10
enter value for empno:7499
```

/

```
enter value for per: 20
enter value for empno: 7900
```

/

```
enter value for per: 15
enter value for empno: 7934
```

Ex-1:

$$\begin{aligned} \text{Total} &= m_1 + m_2 + m_3 \\ &= (m_1 + m_2 + m_3) / 3 \end{aligned}$$

Ex-1:

student

sid	sname	m1	m2	m3	total	avrg
1	A	60	80	80		
2	B	50	30	70		

$$\begin{aligned} \text{total} &= m_1 + m_2 + m_3 \\ \text{avrg} &= (m_1 + m_2 + m_3) / 3 \end{aligned}$$

CREATE TABLE student

```
(
sid number(4),
sname varchar2(10),
m1 number(3),
m2 number(3),
m3 number(3),
total number(3),
avrg number(5,2)
);
```

INSERT INTO student(sid,sname,m1,m2,m3)
VALUES(1,'A',60,80,70);

INSERT INTO student(sid,sname,m1,m2,m3)
VALUES(2,'B',50,30,90);

COMMIT;

Select * FROM student;

UPDATE student SET total=m1+m2+m3,
avrg=(m1+m2+m3)/3;

Assignment:
employee

empno	ename	job	sal	ta	hra	tax	gross
1	A	clerk	5000				

calculate ta, hra, tax & gross?

10% on sal as TA
20% on sal as HRA
5% on sal as TAX
GROSS = sal+TA+HTA-TAX

UPDATE employee SET ta=sal*0.1,
hra=sal*0.1, tax=sal*0.05,
gross=sal+sal*0.1+sal*0.2-sal*0.05;

Transfer all deptno 10 emps to deptno 20:

UPDATE emp SET deptno=20
WHERE deptno=10;

Increase salaries & commissions as following:

If emps are getting commission,
increase 20% on sal as salary
increase 10% on comm as commission

UPDATE emp SET sal=sal+sal*0.2, comm=comm+comm*0.1
WHERE comm is not null;

Increase 15% on sal to the emps who are having more than
40years experience:

**UPDATE emp SET sal=sal+Sal*0.15
WHERE (sysdate-hiredate)/365>40;**

DELETE:

- used to delete the records.
- using this command we can delete:
 - single record
 - a set of records
 - all records
 - using parameters

Syntax:

```
DELETE [FROM] <table_name>  
[WHERE <condition>];
```

Deleting single record:

Delete an emp record whose empno is 7934:

**DELETE FROM emp
WHERE empno=7934;**

Deleting a set of records:

Delete all clerks records:

DELETE FROM emp WHERE job='CLERK';

Deleting all records:

**DELETE FROM emp;
(or)
DELETE emp;**

Deleting records using parameters:

**DELETE FROM emp WHERE empno=&empno;
enter value for empno: 7900**

**/
enter value for empno: 7499**

**/
enter value for empno: 7934**

**Delete all emp records wh are having more than
40years experience:**

**DELETE FROM emp
WHERE (sysdate-hiredate)/365>40;**

Delete all emp records who joined in 1982;

**DELETE FROM emp
WHERE hiredate>='1-jan-1982' AND hiredate<='31-dec-1982';**

(or)

**DELETE FROM emp
WHERE hiredate BETWEEN '1-jan-1982' AND '31-dec-1982';**

(or)

```
DELETE FROM emp
WHERE hiredate LIKE '%82';
```

SQL

DDL	c a d t r f p
DRL	select
DML	insert update delete insert all, merge
DCL	grant, revoke
TCL	commit rollback savepoint

DCL / ACL:

- DCL => Data Control Language.
- ACL => Accessing Control Language
- It deals with data accessibility.
- Using DCL, we can implement table level security.

ORACLE SQL provides following DCL commands:

- GRANT
- REVOKE

GRANT:

- used to give permissions to other users on database objects like tables, views.

Syntax:

```
GRANT <privileges_list> ON <table_name>
TO <user_name>;
```

Revoke:

- Revoke command is used to cancel the permissions from other users

Syntax:

```
REVOKE <privileges_list> ON <table_name>
FROM <user_name>;
```

c##batch4pm:

```
GRANT select ON emp TO c##batch2pm;
```

c##batch2pm:

```
SELECT * FROM c##batch4pm.emp;
--displays the records
```

```
delete from emp;
--ERROR: insufficient privileges
```

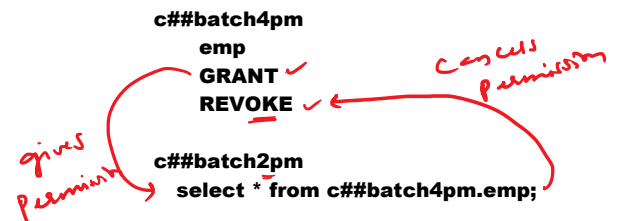
c##batch4pm:

```
GRANT insert,update ON emp TO c##batch2pm;
```

c##batch2pm:

```
UPDATE c##batch4pm.emp SET sal=sal+1000;
14 rows updated.
```

c##batch4pm:



GRANT all ON emp TO c##batch2pm;

**REVOKE insert,update,delete ON emp
FROM c##batch2pm;**

REVOKE all On emp FROM c##batch2pm;

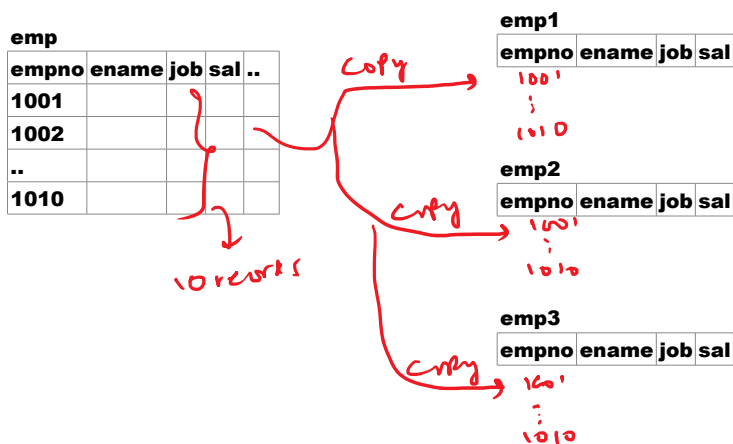
Insert All:

- introduced in Oracle 9i version.
- used to insert multiple records into multiple tables (or) single table.
- Normally, it is used to copy records from existing table & insert into multiple tables.
- It avoids of writing multiple INSERT commands.
- It can be used in 2 styles. They are:
 - Unconditional Insert All
 - Conditional Insert All

Unconditional Insert All:

Syntax:

```
INSERT ALL
  INTO <table_name>[(<column_list>)] VALUES(<value_list>)
  INTO <table_name>[(<column_list>)] VALUES(<value_list>)
  INTO <table_name>[(<column_list>)] VALUES(<value_list>)
  .
  .
  .
<SELECT query>;
```



```
CREATE TABLE emp1
(
  empno number(4),
  ename varchar2(10),
  job varchar2(10),
  sal number(7,2)
);
```

```
CREATE TABLE emp2
(
  empno number(4),
  ename varchar2(10),
  job varchar2(10),
```

```

sal number(7,2)
);

CREATE TABLE emp3
(
empno number(4),
ename varchar2(10),
job varchar2(10),
sal number(7,2)
);

```

```

INSERT ALL
INTO emp1 VALUES(empno,ename,job,sal)
INTO emp2 VALUES(empno,ename,job,sal)
INTO emp3 VALUES(empno,ename,job,sal)
SELECT empno,ename,job,sal FROM emp;

```

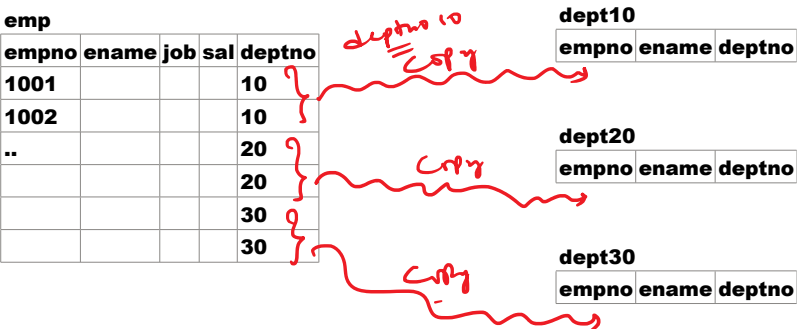
42 rows inserted.

Conditional Insert All:

```

INSERT ALL
WHEN <condition-1> THEN
  INTO <table_name>(<column_list>) VALUES(value_list)
WHEN <condition-2> THEN
  INTO <table_name>(<column_list>) VALUES(value_list)
WHEN <condition-3> THEN
  INTO <table_name>(<column_list>) VALUES(value_list)
.
.
[ELSE
  INTO <table_name>(<column_list>) VALUES(value_list)]
<SELECT query>;

```



```

CREATE TABLE dept10
(
empno number(4),
ename varchar2(10),
deptno number(2)
);

CREATE TABLE dept20
(
empno number(4),
ename varchar2(10),
deptno number(2)
);

CREATE TABLE dept30
(

```



```
empno number(4),
ename varchar2(10),
deptno number(2)
);
```

```
INSERT ALL
WHEN deptno=10 THEN
  INTO dept10 VALUES(empno,ename,deptno)
WHEN deptno=20 THEN
  INTO dept20 VALUES(empno,ename,deptno)
WHEN deptno=30 THEN
  INTO dept30 VALUES(empno,ename,deptno)
SELECT empno,ename,deptno FROM emp;
```

MERGE command:

used to apply the changes of one table to its replica.

Syntax:

```
MERGE INTO <target_table_name> <alias>
USING <source_table_name> <alias>
ON(<condition>)
WHEN matched THEN
  UPDATE query
WHEN not matched THEN
  INSERT query;
```

ORACLE:

RDBMS => create & maintain the database in the form of tables.

SQL  **ORACLE DATABASE**
PL/SQL

SQL:

SQL => Query Language => Queries

5 Sub Languages

DDL	CREATE ALTER DROP TRUNCATE RENAME FLASHBACK PURGE
DRL	SELECT
DML	INSERT DELETE UPDATE INSERT ALL MERGE
DCL	GRANT REVOKE
TCL	COMMIT [SAVE] ROLLBACK SAVEPOINT

$s.cid = t.cid$

Branch

Customer1

cid	cname	ccity
1001	A	Bangalore
1002	B	Goa
1003	C	Mumbai
1004	D	Kolkata
1005	E	Pune

HQ

Customer2

cid	cname	ccity
1001	A	Hyd
1002	B	Delhi
1003	C	Mumbai

Syntax of Merge Command:

```

MERGE INTO <target_table_name> <alias>
USING <source_table_name> <alias>
ON(<condirion>)
WHEN matched THEN
    <Update query>
WHEN not matched THEN
    <INSERT query> ;

```

Example on MERGE:

CREATE TABLE customer1

```

(
cid number(4),
cname varchar2(10),
ccity varchar2(10)
);

```

```

INSERT INTO customer1 VALUES(1,'A','Hyd');
INSERT INTO customer1 VALUES(2,'B','Delhi');
INSERT INTO customer1 VALUES(3,'C','Mumbai');
COMMIT;

```

Copying Table:

CREATE TABLE customer2

```

AS
SELECT * FROM customer1;

```

--with select query result a new table customer2 will be created.

--exact copy of customer1 will be created

```

UPDATE customer1 SET ccity='Bangalore'
WHERE cid=1;

```

```

UPDATE customer1 SET ccity='Goa'
WHERE cid=2;

```

```

INSERT INTO customer1 VALUES(4,'D','Pune');
INSERT INTO customer1 VALUES(5,'E','Chennai');
COMMIT;

```

MERGE:

```
MERGE INTO customer2 t
USING customer1 s
ON(s.cid = t.cid)
WHEN matched THEN
    UPDATE SET t.cname=s.cname, t.ccity=s.ccity;
WHEN not matched THEN
    INSERT VALUES(s.cid,s.cname,s.ccity);
```

```
DELETE FROM emp;      -- deletes all records
(or)
TRUNCATE TABLE emp;  -- deletes all records
```

TRUNCATE	DELETE
• It is DDL Command • It is auto-committed. Truncate = Truncate + commit • We cannot rollback it • We cannot delete single record (or) a group of records. We can delete all records only • We cannot use WHERE clause • Deletes page by page [block by block] • It is faster than DELETE	• It is DML command • It is not auto-committed. • We can rollback it • We can delete single record or group of records or all records. • We can use WHERE clause • Deletes row by row • It is slower than TRUNCATE

SQL => 4GL [4th Generation Language]

It provides readymade commands & readymade functions

Readymade functions are called "Built-In Functions" or "Predefined Functions".

Built-In Functions:

- **Oracle developers already define some functions & placed in Oracle Database. These functions are called "Built-In Functions / Predefined Functions".**
- **Function => Task / Action / Job**
- **Every function performs particular task.**
- **To make our tasks easier ORACLE DEVELOPERS defined "Built-In Functions".**

SQL built-functions can be categorized as following:

- **String Functions [Text Functions / Char Functions]**
- **Aggregate Functions [Group functions]**
- **Math Functions [Number Functions]**
- **Conversion Functions**
- **Date Functions**
- **Miscellaneous Functions**

String Functions [Text Functions / Char Functions]:

lower()	Ltrim()	Instr()	Soundex()
upper()	Rtrim()	Substr()	ASCII()
initcap()	Trim()		Chr()
length()		Replace()	
concat()	Lpad()	Translate()	
	Rpad()	Reverse()	

lower():

- **To convert the string to lower case.**

Syntax:

Syntax:

```
lower(<string>)
```

Exs:

lower('RAJU')	raju
lower('KiRaN')	kiran
lower('RAJ KUMAR')	raj kumar

upper():

Used to convert the string to upper case.

Syntax:

```
upper(<string>)
```

Exs:

upper('ramu')	RAMU
upper('RaJ KuMaR')	RAJ KUMAR

initcap():

It gives every word initial letter as capital.

Syntax:

```
initcap(<string>)
```

Exs:

initcap('RAMU')	Ramu
initcap('RAJ KUMAR')	Raj Kumar

length():

used to find length of the string.

Syntax:

```
length(<string>)
```

Exs:

<code>length('sai')</code>	3
<code>length('ravi teja')</code>	9
<code>length('ramesh')</code>	6

concat():

- **concat** => concatenate [combine]

used to combine 2 strings.

Syntax:

`concat(<string-1>,<string-2>)`

Exs:

<code>concat('raj','kumar')</code>	rajkumar
<code>concat('raj','kumar','varma')</code>	ERROR
<code>concat(concat('raj',' '), 'kumar')</code>	raj kumar

concat('raj', ' ', 'kumar') raj kumar

Display the emp names in lower case:

```
SELECT empno, lower(ename), sal  
FROM emp;
```

Modify emp names to lower case:

```
UPDATE emp SET ename=lower(Ename);
```

Modify emp names to initcap case:

```
UPDATE emp SET ename=initcap(ename);
```

Modify emp names to upper case:

```
UPDATE emp SET ename=upper(ename);
```

Display the emp record whose name is 'BLAKE'
[When we don't know exact case]:

```
SELECT * FROM emp  
WHERE ename='blake';
```

no rows selected

```
SELECT * FROM emp  
WHERE lower(ename)='blake';
```

Display the emp records whose names are having
5 chars:

```
SELECT * FROM emp  
WHERE ename LIKE '_____';  
(or)  
SELECT * FROM emp  
WHERE length(ename)=5;
```

Display the emp records whose names are having
5 or more chars:

```
SELECT * FROM emp  
WHERE length(ename)>=5;
```

Player

pid	fname	lname	Name
1001	Virat	Kohli	Virat Kohli
1002	Rohit	Sharma	

```
ALTER TABLE Player ADD name varchar2(20);
```

```
UPDATE player SET name=  
concat(concat(fname,' '),lname);  
(or)  
UPDATE player SET name= fname || ' ' || lname;
```

REVERSE():
used to reverse the string.

Syntax:

REVERSE(<string>)

Exs:

REVERSE('sai')	ias
REVERSE('ramu')	umar

ASCII():

used to get ASCII value of specified char

Syntax:

ASCII(<char>)

Exs:

ASCII('A')	65
ASCII('Z')	90
ASCII('a')	97
ASCII('z')	122

CHR():

It is used to char of specified ASCII value

Syntax:

CHR(<ASCII_value>)

Exs:

CHR(65)	A
CHR(90)	Z
CHR(97)	a

Soundex():

- used to retrieve the data based on pronunciation**

(sounds).

- When we don't know the exact spelling then we retrieve based on sounds.

Syntax:

Soundex(<string>)

Exs:

Soundex('SMYT') = Soundex('SMITH')

Display the emp record whose name is scott:

**SELECT * FROM emp
WHERE soundex(ename) = soundex('skat');**

Substr():

- used to get sub string from the string.
- sub string => part of the string.

Syntax:

Substr(<string>,<position>[,<no_of_chars>])

1 2 3 4 5 6 7 8 9
r a j k u m a r
-9 -8 -7 -6 -5 -4 -3 -2 -1

Exs:

Substr('raj kumar',5)	kumar
Substr('raj kumar',6)	umar
Substr('raj kumar',1,3)	raj
Substr('raj kumar',1,5)	raj k

1 2 3 4 5 6 7 8 9
r a j k u m a r
-9 -8 -7 -6 -5 -4 -3 -2 -1

Substr('raj kumar',6,3)	uma
Substr('raj kumar',-4)	umar
Substr('raj kumar',-4,3)	uma
Substr('raj kumar',-5)	kumar
Substr('raj kumar',-5,3)	kum

+ve	from left side
-ve	from right side

Instr():

This function is used to get position of sub string.

Syntax:

Instr(<string>,<sub_string>,[<position>,<occurrence>])

position	1
occurrence	1

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
T	h	i	s		i	s		h	i	s		w	i	s	h
-16	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Instr('This is his wish','is')	3
Instr('This is his wish','is',1,2)	6
Instr('This is his wish','is',4)	6
Instr('This is his wish','is',4,2)	10
Instr('This is his wish','are')	0
Instr('This is his wish','is',-1)	14
Instr('This is his wish','is',-1,2)	10
Instr('This is his wish','is',-4)	10
Instr('This is his wish','is',-4,2)	6
Instr('This is his wish','xyz')	0

emp

empno	ename	sal
7369	SMITH	3000
7499	ALLEN	2500

Generate email ids for all emps by taking emp name first 3 chars & empno last 3 chars as username for the domain 'nareshit.com':

```
SELECT empno,ename,  
Substr(ename,1,3) || Substr(empno,-3,3) || '@nareshit.com' as "mail id"  
FROM emp;
```

```
ALTER TABLE emp ADD mail_id varchar2(30);
```

```
UPDATE emp SET mail_id=  
Substr(ename,1,3) || Substr(empno,-3,3) || '@nareshit.com';
```

Display the emp records whose name is started with 'S':

```
SELECT * FROM emp  
WHERE substr(ename,1,1)='S';
```

Display the emp records whose names are ended with 'RD':

```
SELECT * FROM emp  
WHERE substr(ename,-2,2)='RD';
```

**Display the emp records whose name's starting letter
& ending letter are same:**

```
SELECT * FROM emp  
WHERE substr(ename,1,1) = substr(ename,-1,1);
```

**Display the emp records whose names are having
'AM' chars:**

```
ename  
-----  
JAMES  
ALLEN  
SMITH  
ADAMS
```

```
SELECT * FROM emp  
WHERE Instr(ename,'AM')>0;
```

Lpad():

- pad => fill
- used to fill specified char set at left side.

Syntax:

Lpad(<string>,<max_size>[,<char/chars>])

default char [3rd arg]	space
------------------------	-------

Rpad():

used to fill specified char set at right side.

Syntax:

Rpad(<string>,<max_size>[,<char/chars>])

Lpad('ramu',10,'*')	*****ramu
Rpad('kumar',10,'*')	kumar*****
Lpad('ramu',10)	6spacesramu
Rpad('ramu',10)	ramu6spaces
Lpad('*',10,'*')	*****

Amount debited from XXXXXX1234

Acno => 1287651234

SELECT

'amount debited from ' ||

Lpad('X',6,'X') || Substr('1287651234',-4,4)

FROM dual;

Ltrim(), Rtrim() & Trim():

Ltrim():

- Trim => Remove
- Ltrim() is used to remove unwanted chars from left side.

Syntax:

Ltrim(<string>[,<char/chars>])

Rtrim():

- Trim => Remove
- Rtrim() is used to remove unwanted chars from right side.

Syntax:

Rtrim(<string>[,<char/chars>])

Ltrim('@@@ramu@@@','@')	ramu@@@
Rtrim('@@@ramu@@@','@')	@@@ramu

Trim():

used to remove left side or right side or both sides unwanted chars.

Syntax:

Trim(<Leading / Trailing / Both> <char> FROM <string>)

Exs:

Trim(Leading '@' FROM '@@@ravi@@@')	ravi@@@
Trim(Trailing '@' FROM '@@@ravi@@@')	@@@ravi
Trim(Both '@' FROM '@@@ravi@@@')	ravi

Ltrim() & Rtrim() default char is space.

Ltrim(' ramu ')	ramu3spaces
Rtrim(' ramu ')	3spacesramu
Trim(' ramu ')	ramu

Replace():

used to replace search string with replace string.

Syntax:

Replace(<string>,<search_string>,<replace_String>)

Exs:

Replace('sai krishna','sai','rama')	rama krishna
Replace('sai krishna','krishna','teja')	sai teja

Translate():

used to replace search char with corresponding char in replace char set.

Syntax:

Translate(<string>,<search_char_set>,<replace_char_set>)

Exs:

translate('hello','lo','ab')	heaab
replace('hello','lo','ab')	helab

replace()	replaces search string with replace string replace searches for string & replaces the string
translate()	replaces search char with corresponding char in "replace char set". translate searches for char & replaces that char

replace('narayana','na','PQ')	PQrayaPQ
translate('narayana','na','PQ')	PQrQyQPQ

Apply mask chars for salary values:

```
SELECT empno,ename,  
Translate(sal,'0123456789','!@#$$%^&*+-' ) as sal  
FROM emp;
```

Aggregate (Group) Functions:

Aggregate =>

one value is formed from a set of independent values

**Sum()
Avg()
Min()
Max()
Count()**

Sum():

is to find sum of values of a column.

Syntax:

Sum(<column_name>)

Find sum of salaries of all emps:

```
SELECT sum(Sal) FROM emp;
```

Find sum of salaries of deptno 30:

```
SELECT sum(Sal) FROM emp WHERE deptno=30;
```

Find sum of salaries of all managers:

```
SELECT sum(Sal) FROM emp WHERE job='MANAGER';
```

Avg():

used to find average value of a column.

Syntax:

Avg(<column_name>)

Find avrg salary of all emps:

SELECT avg(Sal) FROM emp;

Find avrg salary of deptno 20:

SELECT avg(Sal) FROM emp WHERE deptno=20;

Max():

used to find max value in a column

Syntax:

Max(<column_name>)

Find highest salary in all emps:

SELECT max(Sal) FROM emp;

Find max salary in deptno 30:

SELECT max(Sal) FROM emp WHERE deptno=30;

Min():

used to find min value in a column

Syntax:

Min(<column_name>)

Find min salary in all emps:

SELECT min(Sal) FROM emp;

Find min salary in deptno 30:

SELECT min(Sal) from emp WHERE deptno=30;

Count():

- **used to count no of records or no of values in a column.**
- **It counts result of select query.**
- **It does not count null values.**

Syntax:**Count(<column_name>)****Find no of emps [no of records]:****SELECT count(*) FROM emp;****difference b/w count(*) & count(8):**

count(*)	it counts the no of records gets executed slower
count(8)	it counts no of 8s gets executed faster

How many emps are getting commission:**SELECT count(comm) FROM emp;****Find max salary:****SELECT max(Sal) FROM emp;****Find 2nd max salary:**

sal

5000
9000
8000
7000
4000

**SELECT sal FROM emp
WHERE sal<9000;**

**sal<max sal
find max in those sals**

5000
8000
7000
4000

max = 8000
↓
2nd max sal = 7000

**SELECT max(sal) FROM emp
WHERE sal<(SELECT max(sal) FROM emp);**

sal

5000
9000
8000
7000
4000

Find 3rd max salary:

sal < 2nd max sal
in those salaries find max => 3rd max

sal	

5000	SELECT max(sal) FROM emp
9000	WHERE sal < (
8000	SELECT max(sal) FROM emp
7000	WHERE sal < (SELECT max(sal)
4000	FROM emp));

Conversion Functions:

There are 2 types of conversions. They are:

- Implicit Conversion
- Explicit Conversion

Implicit Conversion:

This conversion will be done by the ORACLE implicitly.

Explicit Conversion:

This conversion will be done by the Built-In Function explicitly.

Ex:

SELECT 100+200 FROM dual;

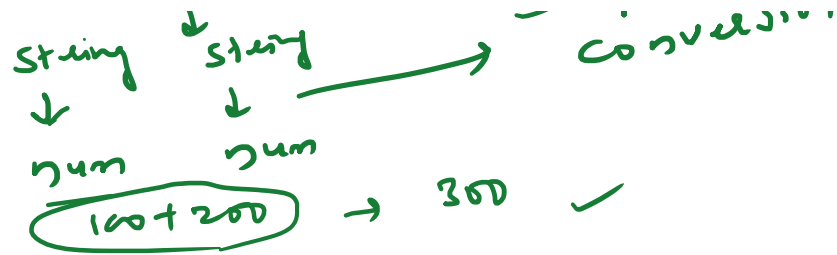
300

SELECT '100' + '200' FROM dual;

300

String String Implicit conversion

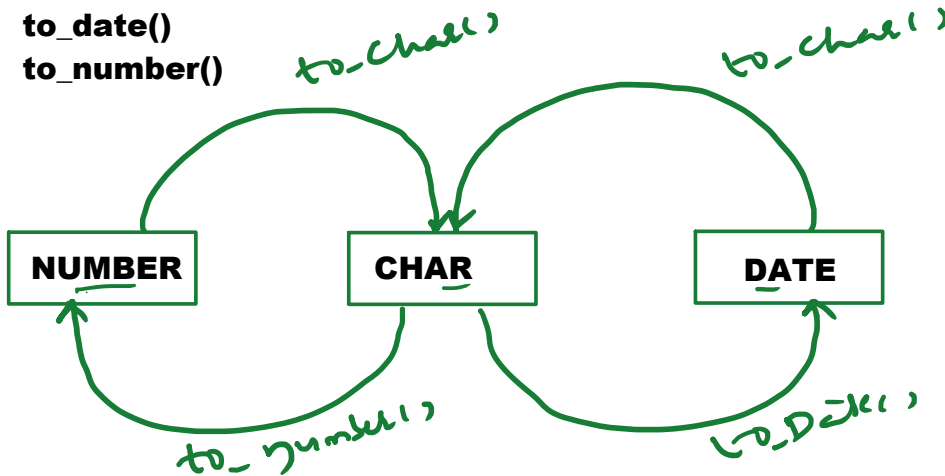
300



to_char()

to_date()

to_number()



to_char() [Date to Char]:

- used to convert date value to string.
- Using this, we can change the date formats like
US Date Format [MM/DD/YYYY],
IND Date Format [DD/MM/YYYY]

Syntax:

to_char(<date>,<format>)

28-jul-22

FORMAT	Purpose	Example
yyyy	year 4 digits	to_char(sysdate,'yyyy') 2022
yyy	year 3 digits	to_char(sysdate,'yyy') 022
yy	year 2 digits	to_char(sysdate,'yy') 22
y	year 1 digit	to_char(sysdate,'yyyy') 2
year / YEAR	year in words	to_char(sysdate,'year') twenty twenty-two to_char(sysdate,'YEAR')

		TWENTY TWENTY-TWO
mm	month 2 digits	to_char(sysdate,'mm') 07
mon / MON	short month name	to_char(sysdate,'mon') jul to_char(sysdate,'MON') JUL
month / MONTH	full month name	to_char(sysdate,'month') july to_char(sysdate,'MONTH') JULY
dd	day num in the month	to_char(sysdate,'dd') 28
ddd	day num in the year	to_char(sysdate,'ddd') 209
d	day num in the week	to_char(sysdate,'d') 5
dy / DY	short weekday name	to_char(sysdate,'dy') thu
day / DAY	full weekday name	to_char(sysdate,'day') thursday
q	quarter number q1 => jan-mar q2 => apr-jun q3 => jul-sep q4 => oct-dec	to_char(sysdate,'q') 3
cc	century number	to_char(sysdate,'cc') 21
HH / HH12	hours part	
HH24	hours part	
MI	minutes part	
SS	seconds part	
FF	fractional seconds	
AM / PM	AM or PM	
BC / AD	BC or AD	

Display current system time:

```
SELECT to_char(sysdate,'hh:mi:ss AM') from dual;
```

Display the emp records who joined in 1982:

```
SELECT empno,ename,hiredate  
FROM emp  
WHERE hiredate BETWEEN '1-jan-1982' AND '31-dec-1982;
```

(or)

```
SELECT empno,ename,hiredate  
FROM emp  
WHERE to_char(hiredate,'yyyy')=1982;
```

Display the emp records who joined in 1980, 1982, 1983:

```
SELECT empno,ename,hiredate  
FROM emp  
WHERE to_char(hiredate,'yyyy') IN(1980,1982,1983);
```

Display the emp records who joined in 4th quarter:

4 th quarter => oct - dec

```
SELECT empno,ename,hiredate  
FROM emp  
WHERE to_char(hiredate,'q')=4;
```

Display the emp records who joined in 1st & 4th quarters:

```
SELECT empno,ename,hiredate  
FROM emp  
WHERE to_Char(hiredate,'q') IN(1,4);
```

Display the hiredates in IND Date Format [dd/mm/yyyy]:

```
SELECT empno,ename,
to_char(hiredate,'dd/mm/yyyy') as hiredate
FROM emp;
```

Display the hiredates in US Date Format [mm/dd/yyyy]:

```
SELECT empno,ename,
to_char(hiredate,'mm/dd/yyyy') as hiredate
FROM emp;
```

Display the emp records who joined on Sunday:

```
SELECT empno,ename,hiredate
FROM emp
WHERE to_Char(hiredate,'d')=1;
(or)
SELECT empno,ename,hiredate
FROM emp
WHERE to_Char(hiredate,'dy')='sun';
(or)
```

```
SELECT empno,ename,hiredate
FROM emp
WHERE to_Char(hiredate,'day')='sunday';
```

sunday3spaces
monday3spaces
tuesday2spaces
Wednesday
thursday1space

no rows selected

'sunday...' = 'sunday'
↑ = ↓

```
SELECT empno,ename,hiredate
FROM emp
WHERE Rtrim(to_Char(hiredate,'day'))='sunday';
```

to_char() [number to string]:

- to_char() can be used to convert number to string
- to apply currency formats to numbers, it can be used

Syntax:

to_char(<number>[,<format>])
number → string

Exs:

to_char(123)	123
to_char(123.45)	123.45

\$5,000.00

FORMAT	purpose
L	currency symbol [\$]
C	currency name [USD]
,	thousand separator
.	decimal point
9 / 0	digit

Display 5000 number as \$5000.00:

SELECT to_Char(5000,'L9999.99') FROM dual;

\$5000.00

Display 5000 number as USD5,000.00:

SELECT to_Char(5000,'C9,999.99') FROM dual;

USD5,000.00

Display all emp records along with salaries. Display salaries using currency symbol \$:

**SELECT empno,ename,
to_char(sal,'L99,999.99') as salary
FROM emp;**

to_Date():
used to convert string to date.

Syntax:
to_date(<string>[,<format>])

Exs:

to_date('25-dec-2019')	25-DEC-19
to_date('25 december, 2019')	25-DEC-19
to_Date('25/12/2019')	ERROR
to_date('25/12/2019','dd/mm/yyyy')	25-DEC-19

Extract year from the date 17-sep-2018:

SELECT to_char(sysdate,'yyyy') FROM dual;
2022

SELECT to_char('17-sep-2018','yyyy') FROM dual;
ERROR:

↓
string

SELECT to_char(to_Date('17-sep-2018'),'yyyy')
FROM dual;

↓
string
↓
date

ON which weekday India got Independence?

15-aug-1947

dy	sun
day	Sunday
d	1

SELECT to_Char(to_Date('15-Aug-1947'),'day')
FROM dual;

Find the weekday name on which weekday
sachin born:
24-apr-1973

SELECT to_char(to_Date('24-apr-1973'),'day')
FROM dual;

to_Number():

- used to convert string to number.
- String must be numeric string.

Syntax:**to_Number(<string>[,<format>)****Exs:**

to_Number('123')	123
to_Number('123.45')	123.45
to_Number('\$5,000.00')	ERROR
to_Number('\$5,000.00','L9,999.99')	5000
to_Number('USD5000.00','C9999.99')	5000

Math Functions [Number Functions]:

power()	sign()	mod()
sqrt()	abs()	ceil()
sin()		floor()
cos()		trunc()
tan()		round()

power():**used to find power value****Syntax:****power(number,power)****Exs:**

power(2,3)	8
power(5,2)	25

sqrt():**used to find square root value.**

Syntax:**sqrt(<number>)****Exs:**

sqrt(100)	10
sqrt(81)	9

sign():

- **used to know sign of number.**
- **It can be used to check whether the number is +ve or -ve or zero.**
- **It returns 1, if +ve**
- **It returns -1, if -ve**
- **It returns 0, if zero**

Syntax:**sign(number)****Exs:**

sign(25)	1
sign(-25)	-1
sign(0)	0

abs():

- **Absolute value => Non-Negative**
- **It is used to get absolute value.**

Syntax:**abs(number)****Exs:**

abs(25)	25
abs(-25)	25

sin():**used to get sine value of specified angle.****cos():****used to get cosine value of specified angle.****tan():**

used to get tangent value of specified angle.

Syntaxes:

sin(angle)
cos(angle)
tan(angle)

NOTE:

angle must be specified in radians

sin($90 \times 3.14/180$)
cos($0 \times 3.14/180$)
tan($45 \times 3.14/180$)

mod():

used to get remainder value.

Syntax:

mod(number, divisor)

Ex:

mod(10,7)	3
mod(5,2)	1

ceil():

used to get upper integer value.

Syntax:

ceil(number)

Ex:

ceil(123.5678)	123 & 124 124
ceil(456.23892)	456 & 457 457

floor():
used to get lower integer value

Syntax:
floor(number)

Exs:

floor(123.56789)	123 & 124 123
floor(786.2345)	786 & 787 786

Trunc():

- **Truncate => Remove**
- **It is used to remove the decimal places**

Syntax:
Trunc(<number>[,<no_of_decimal_places>])

Exs:

Trunc(123.456789)	123
Trunc(123.456789,2)	123.45
Trunc(56.2398567,3)	56.239

2nd argument can be specified as -ve also

-1	rounds in the 10s
-2	rounds in the 100s
-3	rounds in the 1000s

Note:

Trunc() always gives lower value

Trunc(123.45678,-1)	120 & 130 120
Trunc(757.89234,-1)	750 & 760 750
Trunc(757.89234,-2)	700 & 800 700
Trunc(4567.2345,-3)	4000 & 5000

4000

Round():

- It is used to get rounded value.
- If value is avrg or above avrg, it takes upper value.
- If value is below avrg, it takes lower value.

Trunc()	percentage	Round()
56.78	56.78 → 56	56.78 → 57
67.82	67.82 → 67	67.82 → 68
65.23	65.23 → 65	65.23 → 65
78.89	78.89 → 78	78.89 → 79
89.23	89.23 → 89	89.23 → 89

Exs:

Round(56.789)	57
Round(56.234)	56
Round(56.5)	57
Round(56.234567,2)	56.23
Round(56.237567,2)	56.24

-1	rounds in 10s
-2	rounds in 100s
-3	rounds in 1000s

Round(123.6789,-1)	120 & 130 => avrg 125 120
Round(127.6789,-1)	120 & 130 => avrg 125 130
Trunc(123.6789,-1)	120 & 130 => avrg 125 120

Trunc(127.6789,-1)	120 & 130 => avrg 125 120
Round(567.34567,-2)	500 & 600 => 550 600
Round(538.34567,-2)	500 & 600 => 550 500

Date Functions:

sysdate

systimestamp

Add_Months()

Last_Day()

Next_Day()

Months_Between()

sysdate:

used to get current system date.

Syntax:

sysdate

Display current system date:

SELECT sysdate FROM dual;

Display current system time:

**SELECT to_char(sysdate,'HH:MI:SS AM')
FROM dual;**

systimestamp:

it is used to get current system date & time.

Syntax:

systimestamp

Display current system date & time:

SELECT systimestamp FROM dual;

Add_Months():

used to add months to a date or subtract months from a date.

Syntax:

Add_Months(<date>,<no_of_months>)

Add 2 days to system date:

1-aug-2022

SELECT sysdate+2 FROM dual;

3-aug-22

Add 2 months to system date:

SELECT Add_Months(sysdate,2) FROM dual;

1-oct-22

Add 2 years to system date:

SELECT Add_Months(sysdate,2*12) FROM dual;

1-aug-24

Subtract 2 days from sysdate:

SELECT sysdate-2 FROM dual;

30-jul-22

Subtract 2 months from sysdate:

SELECT Add_Months(sysdate,-2) FROM dual;

1-jun-22

Subtract 2 years from sysdate:

SELECT Add_Months(sysdate,-2*12) FROM dual;

1-aug-20

INSERT INTO emp(empno,ename,hiredate)

VALUES(2001,'A',sysdate);

**INSERT INTO emp(empno,ename,hiredate)
VALUES(2002,'B',sysdate-1);**

**INSERT INTO emp(empno,ename,hiredate)
VALUES(2003,'C',Add_months(sysdate,-1));**

**INSERT INTO emp(empno,ename,hiredate)
VALUES(2004,'D',Add_months(sysdate,-1*12));**

Display the emp records who joined today:

**SELECT empno,ename,hiredate FROM emp
WHERE hiredate = sysdate;**

no rows selected.

Note:

Date also contains time value.

When we inserted emp record at that time was: 4:35

Now the time is: 4:45

1-aug-22 4:35 = 1-aug-22 4:45 => False =>

no rows selected

Remove time value from date & compare.

To Remove time value from date value, use Trunc()

Trunc() Function removes time value from date & time.

**SELECT empno,ename,hiredate
FROM emp
WHERE trunc(hiredate) = trunc(sysdate);**

Display the emp records who joined yesterday:

**SELECT empno,ename,hiredate FROM emp
WHERE trunc(hiredate) = trunc(sysdate-1);**

Display the emp records who joined 1 month ago from today's date:

```
SELECT empno,ename,hiredate FROM emp  
WHERE trunc(hiredate) = trunc(Add_Months(sysdate,-1));
```

Display the emp records who joined 1 year ago from today's date:

```
SELECT empno,ename,hiredate FROM emp  
WHERE trunc(hiredate) = trunc(Add_months(sysdate,-12));
```

sales

dateid	amount
1-jan-2018	75000
2-jan-2018	100000
3-jan-2018	90000
.	
.	
1-aug-2022	120000

today's sales?

yesterday sales?

1 month ago sales?

1 year ago sales?

Last_Day():

used to get last date in the month.

Syntax:

Last_Day(<date>)

Exs:

1-aug-22

Last_Day(sysdate)	31-aug-22
Last_Day('17-feb-2022')	28-feb-22
Last_Day('17-feb-2020')	29-feb-22

Next_day():

Used to get next date based on the weekday name

from specified date.

Syntax:

Next_day(<date>,<week_day>)

Ex:

find next Friday date from today's date:

1-aug-22 => monday

SELECT next_Day(sysdate,'fri') FROM dual;

5-aug-22 => friday

Find next Sunday's date from today's date:

SELECT next_Day(sysdate,'sun') FROM dual;

7-aug-22

Months_Between():

used to get number of months between 2 date values.

Syntax:

Months_Between(date-1,date-2)

Exs:

Calculate experience of all emps:

**SELECT empno,ename,
Trunc((sysdate-hiredate)/365) as experience
FROM emp;
(or)**

**SELECT empno,ename,
Trunc(Months_Between(sysdate,hiredate)/12) as experience
FROM emp;**

Calculate age of Sachin [DOB: 24-apr-1973]

**SELECT
Trunc(months_between(sysdate,'24-apr-1973')/12) as age
FROM dual;**

Miscellaneous Functions:

NVL()

NVL2()

Rank()

Dense_Rank()

Decode()

Greatest()

Least()

NVL(): [NullVaLue]

used to replace null value with some other value.

Syntax:

NVL(<value-1>,<value-2>)

If first value is not null, NVL() returns 1st value

If first value is null, NVL() returns 2nd value

Ex:

NVL(100,200)	100
NVL(null,200)	200

If NULL is participated in operation then it returns NULL value

100+200-50+400-null	null
100+200+null	null

Calculate total salary gain by emp [sal+comm]:

**SELECT empno,ename,sal,comm,
sal+NVL(comm,0) as "total salary" FROM emp;**

**Display comm value as N/A [not applicable] to the emps
whose comm is null:**

**SELECT empno,ename,sal,
NVL(comm,'N/A') as comm FROM emp;**

ERROR: data type mismatched

comm => num type

'N/A' => char type

SELECT empno,ename,sal,

NVL(to_char(comm),'N/A') as comm FROM emp;

Student

NVL(to_char(maths),'AB')

sid	sname	maths
1001	A	67
1002	B	
1003	C	55
1004	D	

→ AB
→ AB

NVL2():

used to replace nulls & not nulls.

Syntax:

NVL2(value-1,value-2,value-3)

It returns 2nd value if first value is not null

It returns 3rd value if first value is null

Exs:

NVL2(100,200,300)	200
NVL2(null,200,300)	300

Update comm as following:

set comm as 500 those who are not getting comm

increase 1000 rupees comm to the emps who are getting comm

comm

null

300

500

null

1400
null
null

UPDATE emp SET comm=NVL2(comm,comm+1000,500);

NVL()	NVL2()
• used to replace nulls • can take 2 args	• used to replace nulls & not nulls • takes 3 args

Rank():

used to apply to ranks to records according particular column's order.

Syntax:

Rank() Over(Order By <column> ASC/DESC)

Dense_Rank():

used to apply to ranks to records according particular column's order.

Syntax:

Dense_Rank() Over(Order By <column> ASC/DESC)

Ex:

marks	Rank	Dense Rank
678	3	4
930	1	1
890	2	2
550	4	3

890	3	2
550	9	5
930	1	1
890	3	2
678	7	4
890	3	2
745	6	3

Apply ranks to emp records according to salary descending order:

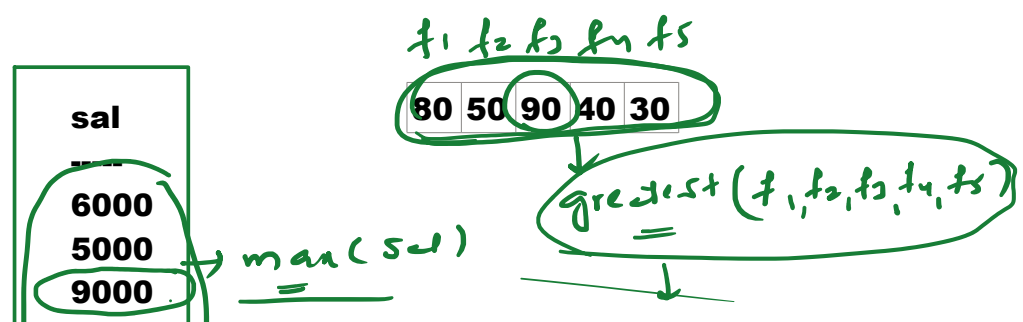
**SELECT empno,ename,sal,
Rank() Over(Order BY sal DESC) as Rank
FROM emp;**

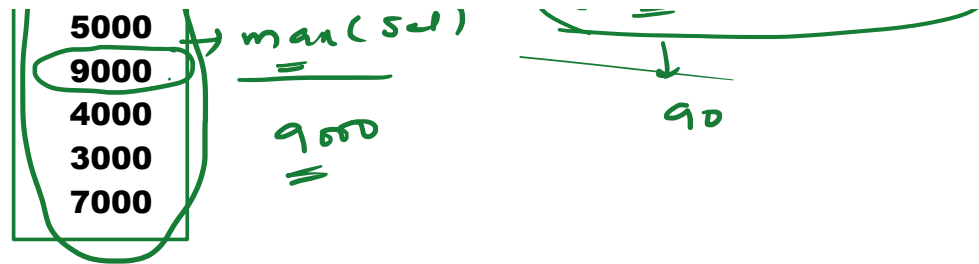
**SELECT empno,ename,sal,
Dense_Rank() Over(Order BY sal DESC) as Rank
FROM emp;**

Rank()	It does not follow the sequence in ranking
Dense_Rank()	It follows sequence in ranking

Greatest():
used to find max value in horizontal values.

Max():
used to find max value in vertical values.





SELECT max(sal) FROM emp;

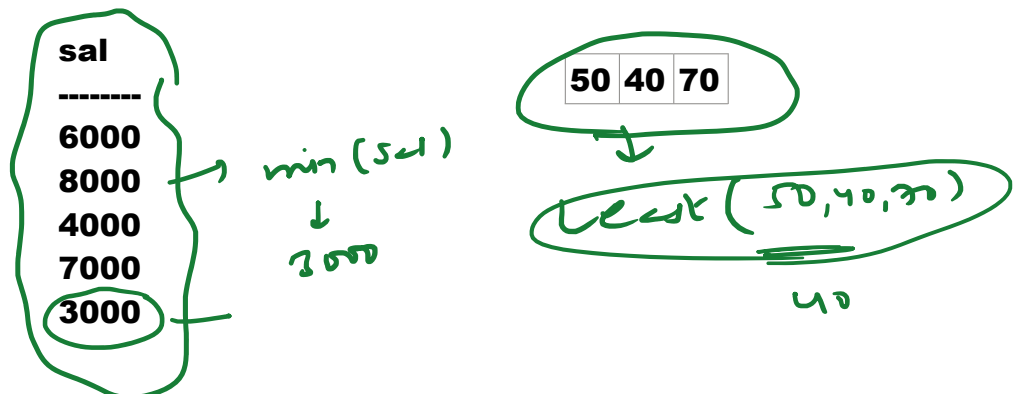
**SELECT greatest(10,90,20,30,56,78,99,44,81)
FROM dual;
99**

Least():

used to find min value in horizontal values.

Min():

used to find min value in vertical values.



Decode():

used to implement IF .. THEN .. ELSE in SQL.

Syntax:

**Decode(<column>,
<value>,<return_expression>,
<value>,<return_expression>,
<value>,<return_expression>,
..... ,
..... ,
else_Expression)**

Example on Decode():

Display all emp records with job titles as following:

PRESIDENT => Big Boss

MANAGER => Boss

CLERK => Worker

OTHERS => EMployee

```
SELECT empno,ename,job,  
Decode(job,  
'PRESIDENT','BIG BOSS',  
'MANAGER','BOSS',  
'CLERK','WORKER',  
'EMPLOYEE') as Job  
FROM emp;
```


Clauses

Tuesday, August 2, 2022 4:52 PM

Clauses in SELECT command:

Clause is a part of query.

ENGLISH
Sentences
Words

SQL
Queries
Clauses

Syntax of SELECT command:

Default

```
SELECT [DISTINCT / ALL] <column_list>
FROM <table_list>
[WHERE <condition>]
[GROUP BY <group_column_list>]
[HAVING <group_condition>]
[ORDER BY <column_name> ASC / DESC];
```

SELECT query has following clauses:

- **SELECT**
- **FROM**
- **WHERE**
- **DISTINCT**
- **ORDER BY**
- **GROUP BY**
- **HAVING**

SELECT	used to specify column list
• FROM	used to specify table list
WHERE	used to write conditions on rows

```
SELECT empno,ename,sal
FROM emp
WHERE sal>3000;
```

Execution Order:

FROM

WHERE
GROUP BY
HAVING
SELECT
DISTINCT
ORDER BY

SELECT empno,ename,sal
FROM emp
WHERE sal>3000;

FROM emp:

emp

empno	ename	job	sal
1001	A		6000
1002	B		2000
1003	C		5000
1004	D		2500
1005	E		3500

sal > 3000
6000 - T ✓
2000 - F X
5000 - T ✓
2500 - F X
3500 - T ✓

WHERE sal>3000:

empno	ename	job	sal
1001	A		6000
1003	C		5000
1005	E		3500

SELECT empno,ename,sal:

empno	ename	sal
1001	A	6000
1003	C	5000
1005	E	3500

DISTINCT:

- **DISTINCT** clause used to avoid duplicate rows.
- It means, it gives distinct rows.

Display the job titles offered by company:

emp

job

SELECT DISTINCT job FROM emp;

**CLERK
MANAGER
MANAGER
CLERK
SALAESMAN
CLERK
MANAGER
SALESMAN
SALESMAN**

job

**CLERK
MANAGER
SALESMAN**

Display the deptnos available in company:

deptno

**20
30
30
20
20
20
10
10
30
20
10**

SELECT DISTINCT deptno FROM emp;

deptno

**10
20
30**

ORDER BY:

is used to arrange the records in ascending or descending order according to specific column / columns.

Display all emp records. Display emp names in alphabetical order:

```
SELECT empno,ename,sal
FROM emp
ORDER BY ename;          --default ASC
```

(or)

```
SELECT empno,ename,sal
FROM emp
ORDER BY ename ASC;
```

(or)

```
SELECT empno,ename,sal
FROM emp
ORDER BY 2;
```

① ② ③

Display the emp records in descending order according to salary:

```
SELECT empno,ename,sal
FROM emp
ORDER BY sal DESC;
```

(or)

```
SELECT empno,ename,sal
FROM emp
ORDER BY 3 DESC;
```

Display the emp records in ascending order according to deptno. If dept num is same check with sal. Arrange salaries in descending order:

[With in the dept arrange salaries in descending order]

```

SELECT empno,ename,deptno,sal
FROM emp
ORDER BY deptno ASC,sal DESC;

```

First priority for deptno.

If deptno is same then only it checks with salary.

GROUP BY:

- used to group the records according to specific column / columns.

Example:

emp

Table detailed data

empno	ename	job	deptno	sal
1001	A	clerk	10	5000
1002	B	manager	10	9000
1003	C	clerk	20	7000
1004	D	clerk	20	6000
1004	E	manager	20	12000
1005	F	manager	30	15000
1006	G	clerk	30	8000

Group By deptno

Summarized data

deptno	sum(Sal)
10	14000
20	25000
30	23000

job	NO_OF_EMPS
CLERK	4
MANAGER	3

deptno	no_of_emps
10	2
20	3
30	2

GROUP BY:

- used to group the records according to specific column / columns.
- If column value is same, it will be treated as one group.
- It is used get summarized data from detailed data.
- We can apply aggregate functions on group of records.

Examples on GROUP BY:

Find dept wise sum of salaries:

deptno	sum_of_Sal
10	10000
20	15000
30	9000

```
SELECT deptno,sum(sal) as sum_of_Sal  
FROM emp  
GROUP BY deptno  
ORDER BY 1;
```

Find dept wise no of emps:

deptno	no_of_emps
10	3
20	5
30	6

```
SELECT deptno, count(*) as no_of_emps  
FROM emp  
GROUP BY deptno  
ORDER BY deptno;
```

Find dept wise max salary & min salary:

deptno	max_Sal	min_Sal
10	6000	2000
20	9000	3000
30	8000	1500

```
SELECT deptno,max(Sal) as max_Sal,  
min(Sal) as min_sal  
FROM emp  
GROUP BY deptno  
ORDER BY deptno;
```

Display year wise no of emps joined in organization:

year	no_of_emps
1980	1
1981	7
1982	2
1983	3

```
SELECT to_char(hiredate,'yyyy') as year,  
count(*) as no_of_emps  
FROM emp  
GROUP BY year;
```

ERROR: "year" invalid identifier

```
FROM  
GROUP BY  
SELECT
```

can we use column alias in GROUP BY?

NO. GROUP BY will be executed before SELECT.

can we use column alias in ORDER BY?

Yes. ORDER BY gets executed after SELECT.

```
SELECT to_char(hiredate,'yyyy') as year,  
count(*) as no_of_emps  
FROM emp  
GROUP BY to_char(hiredate,'yyyy')  
ORDER BY year;
```

Find quarter wise no of emps joined in organization:

quarter	no_of_emps
1	3
2	5
3	4
4	2

```

SELECT to_char(hiredate,'q') as quarter,
count(*) as no_of_emps
FROM emp
GROUP BY to_char(hiredate,'q')
ORDER BY quarter;

```

Find job wise sum of salaries:

job	sum_of_sal
clerk	10000
manager	20000
salesman	15000
analyst	12000

```

SELECT job,sum(sal) as sum_of_sal
FROM emp
GROUP BY job;

```

Find job wise max sal & min sal:

```

SELECT job,max(Sal) as max_sal,
min(Sal) as min_Sal
FROM emp
GROUP BY job;

```

HAVING clause:

- **used to write conditions on groups [group of records]**
- **It cannot be used without GROUP BY.**
- **It must be followed by GROUP BY.**
- **HAVING clause is applied on result of GROUP BY.**

WHERE => condition => applied on rows
Aggregate functions cannot be used in WHERE clause

HAVING => condition => applied on groups
Aggregate functions can be used in HAVING

clause

Display the deptnos which are spending more than 15000 amount on their emps:

deptno	sum(Sal)
10	11500
20	20000
30	11200

GROUP BY deptno
HAVING sum(sal)>15000

x 11500 > 15000 - F
20000 > " - T
x 11200 > 15000 - F

20	20000
----	-------

SELECT deptno,sum(Sal)
FROM emp
GROUP BY deptno
HAVING sum(Sal)>15000
ORDER BY deptno;

Display the deptnos which are having 5 or more emps:

SELECT deptno,count(*)
FROM emp
GROUP BY deptno
HAVING count(*)>=5;

deptno	count(*)
10	3
20	5
30	6

HAVING count(*)>=5

x 3 >= 5 - F
✓ 5 >= 5 - T
✓ 6 >= 5 - T

Group by deptno

Differences b/w WHERE & HAVING:

WHERE	HAVING
• WHERE clause condition will be applied on every row • We cannot use Aggregate functions in WHERE clause • It can be used without GROUP BY • It will be executed before GROUP BY	• HAVING clause condition will be applied on every group • We can use Aggregate functions in HAVING clause • It cannot be used without GROUP BY • It will be executed after GROUP BY

FROM	used to specify table names
WHERE	to specify conditions on rows
GROUP BY	used to group the records according to specific column / columns
HAVING	to specify condition on groups
SELECT	used to specify column names
DISTINCT	used to avoid duplicate rows
ORDER BY	used to arrange records in ASC or DESC order according to specific column or columns

emp

empno	ename	sal	deptno
1001	A	..	20
1002	B	..	20
1003	C	..	30
1004	D	..	10
1005	E	..	10
1006	F	..	30
1007	G	..	
1008	H	..	

dept

deptno	dname	loc
10	ACCOUNT	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

Join:

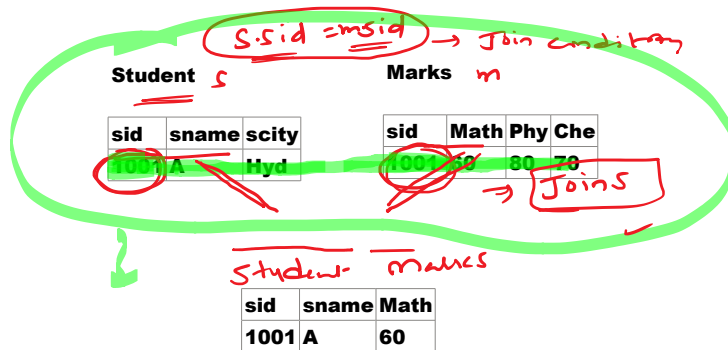
- Join => connect / combine / link

College DB

Student
Marks
Fee
Staff

Bank DB

Customer
Transaction
Branch
Product
Staff

**Joins:**

- Join => Connect / Combine / Link
- Join is an operation that combines one table records with another based on Join Condition.
- Join Condition decides which record in one table should be joined with which record in another table.
- Joins Concept is used to retrieve the data from multiple tables.

Types of Joins:

- Equi Join / Inner Join
- Outer Join
 - Left Outer Join
 - Right Outer Join
 - Full Outer Join
- Non-Equi Join
- Self Join
- Cross Join / Product Join / Cartesian Join

Equi Join / Inner Join:

- If Join Operation is performed based on equality condition then it is called "Equi Join".
- Common column is required for Equi Join.
- Common column name need not to be same.

e.deptno = d.deptno → Join condition

emp e

empno	ename	sal	deptno
1001	A	..	20
1002	B	..	20
1003	C	..	30
1004	D	..	10
1005	E	..	10
1006	F	..	30
1007	G	..	
1008	H	..	

dept d

deptno	dname	loc
10	ACCOUNT	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

ename	sal	dname	loc
emp e		dept d	

From Oracle 9i version onwards we can write Join Query in 2 styles:

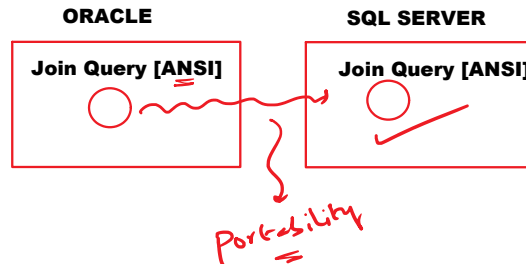
- Oracle Style / Native Style
- ANSI style => Best way => Portable

NOTE:

- To separate 2 table names we use , in ORACLE style WHEREAS we use keyword in ANSI style.
- In ORACLE style, We write Join Condition in WHERE clause WHEREAS In ANSI style we write Join condition in ON clause or USING clause.

In ANSI style we can write join condition in:

- ON clause [Best way]
- USING clause



Display the emp details along with dept details:

ORACLE style:

```
SELECT e.ename,e.sal,d.dname,d.loc
FROM emp e, dept d
WHERE e.deptno = d.deptno;
```

ANSI Style:

```
SELECT e.ename,e.sal,d.dname,d.loc
FROM emp e INNER JOIN dept d
ON e.deptno=d.deptno;
```

(or)

```
SELECT e.ename,e.sal,d.dname,d.loc
FROM emp e INNER JOIN dept d
USING(deptno);
```

*ORACLE rewrites as
ON e.deptno = d.deptno*

- ON clause gets executed faster than USING clause. Because, USING clause will be implicitly converted to ON clause only.
- For USING clause, Common column name must be same. Otherwise, it is not possible.

Outer Join:

- Inner Join can give matched records only. It cannot give unmatched records. To get unmatched records also, use Outer Join

Outer Join = matched + unmatched records

Inner Join = matched record only

- Outer Join Operator is used to write Outer Join in ORACLE style.
- Outer Join Operator Symbol: (+)

emp

empno	ename	sal	deptno
1001	A	..	20
1002	B	..	20
1003	C	..	30
1004	D	..	10
1005	E	..	10

dept

deptno	dname	loc
10	ACCOUNT	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

1006	F	..	30
1007	G	..	
1008	H	..	

There are 3 sub types in Outer Join:

- Left Outer Join
- Right Outer Join
- Full Outer Join

Left Outer Join:

Left Outer = matched + unmatched from left table

- It gives matched records & unmatched records from left side table.
- IN ANSI style, we use "LEFT OUTER JOIN / LEFT JOIN" keyword.
- In Oracle Style, we specify (+) at right side.

Example on Left Outer Join:

Display emp records along with dept details. Also display the emp records to whom deptno is not assigned:

ORACLE style:

```
SELECT e.ename,e.sal,d.dname,d.loc
FROM emp e,dept d
WHERE e.deptno = d.deptno(+);
```

ANSI style:

```
SELECT e.ename,e.sal,d.dname,d.loc
FROM emp e LEFT OUTER JOIN dept d
ON e.deptno = d.deptno;
```

NOTE:

In Oracle Style, join condition decides left & right table

Ex:

```
WHERE e.deptno = d.deptno(+)
emp e => left table
dept d => right table
```

```
WHERE d.deptno = e.deptno(+)
dept d => left table
emp e => right table
```

In ANSI style, keyword decides left table & right table.

Ex:

```
emp e LEFT OUTER JOIN dept d
emp e => left table
dept d => right table
```

```
dept d LEFT OUTER JOIN emp e
dept d => Left table
emp e => right table
```

Right Outer Join:

Right Outer = matched + unmatched from right table

- **Right Outer Join** gives matched records & unmatched records from right side table.
- In **ORACLE** style use (+) symbol at left side.
- In **ANSI** style use "RIGHT OUTER JOIN / RIGHT JOIN" keyword.

Example on Right Outer Join:

Display emp records along with dept details. Also display the dept names in which emps are not existed:

ORACLE style:

```
SELECT e.ename,e.sal,d.dname,d.loc
FROM emp e,dept d
WHERE e.deptno(+) = d.deptno;
```

ANSI style:

```
SELECT e.ename,e.sal,d.dname,d.loc
FROM emp e RIGHT OUTER JOIN dept d
ON e.deptno = d.deptno;
```

Full Outer Join:

Full Outer = matched + unmatched from left & right

- It gives matched records, unmatched records from left table & right table.

e.deptno = d.deptno(+)	Left Outer Join
e.deptno(+) = d.deptno	Right Outer Join
e.deptno(+) = d.deptno(+)	ERROR
LEFT OUTER JOIN UNION RIGHT OUTER JOIN	Full Outer Join

A = { 1,2,3,4,5} Left Outer Join = matched + unmatched from left

B = { 6,7,1,2,3} Right Outer Join = matched + unmatched from right

AUB = {1,2,3,4,5,6,7}

Full Outer Join:

Left Outer U Right Outer = matched + unmatched from left + unmatched from right

Example on Full Outer Join:

Display emp details along with dept details. Also display the emp records to whom deptno is not assigned. Also display the dept names in which emps are not existed:

ORACLE style:

```
SELECT e.ename,e.sal,d.dname,d.loc
FROM emp e, dept d
WHERE e.deptno = d.deptno(+)
UNION
SELECT e.ename,e.sal,d.dname,d.loc
FROM emp e, dept d
WHERE e.deptno(+) = d.deptno;
```

Handwritten notes: } Left outer, UNION, } Right outer

ANSI style:

```
SELECT e.ename,e.sal,d.dname,d.loc
FROM emp e FULL OUTER JOIN dept d
ON e.deptno = d.deptno;
```

INNER JOIN	matched
OUTER JOIN	matched + unmatched

LEFT OUTER	matched + unmatched from left
RIGHT OUTER	matched + unmatched from right
FULL OUTER	matched + unmatched from Left & Right

e.deptno = d.deptno =>Equi Join

Non-Equi Join:

If Join operation is performed based on other than equality condition then it is called "Non-Equi Join".

Ex:

e.deptno != d.deptno
e.deptno > d.deptno
e.deptno < d.deptno
e.deptno >= d.deptno

Non-Equi Join:

e.deptno != d.deptno
e.deptno > d.deptno
e.deptno < d.deptno
e.deptno >= d.deptno

Non-Equi Join → e.sal BETWEEN s.losal AND s.hisal

empno	ename	sal	deptno
1001	A	6000	20
1002	B	1400	20
1003	C	2500	30
1004	D	3500	10

GRADE	LOSAL	HISAL
1	700	1500
2	1501	2000
3	2001	3000
4	3001	4000
5	4001	9999

S. losal S. hisal

EMPNO	ENAME	SAL	grade
-------	-------	-----	-------

Display emp details with salary grades
{Non-Equi Join}:

ORACLE style:

```
SELECT e.ename,e.sal,s.grade
FROM emp e, salgrade s
WHERE e.sal BETWEEN s.losal AND s.hisal;
```

ANSI style:

```
SELECT e.ename,e.sal,s.grade
FROM emp e INNER JOIN salgrade s
ON e.sal BETWEEN s.losal AND s.hisal;
```

Self-Join:

- It can be also called as "Recursive Join".

e.mgr = m.empno

empno	ename	job	sal	mgr
1001	A	manager	9000	1010
1002	B	clerk	4000	1001
1003	C	salesman	5000	1001
1004	D	manager	10000	1010
1005	E	clerk	6000	1004
1006	F	analyst	5000	1004
1010	G	president	25000	

empno	ename	job	sal	mgr
1001	A	manager	9000	1010
1002	B	clerk	4000	1001
1003	C	salesman	5000	1001
1004	D	manager	10000	1010
1005	E	clerk	6000	1004
1006	F	analyst	5000	1004
1010	G	president	25000	

Self-Join:

- It can be also called as "Recursive Join".
- In this, one table record will be joined with another record in same table.

- If table is joined to itself then it is called "Self-Join"

Display the emp details along with manager details:

ename	sal	mgr_name	mgr_sal
-------	-----	----------	---------

ORACLE style:

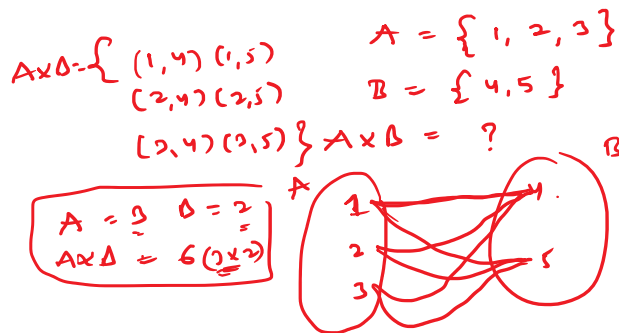
```
SELECT e.ename emp_name,e.sal emp_sal,
m.ename mgr_name,m.sal mgr_sal
FROM emp e,emp m
WHERE e.mgr=m.empno;
```

ANSI style:

```
SELECT e.ename emp_name,e.sal emp_sal,
m.ename mgr_name,m.sal mgr_sal
FROM emp e INNER JOIN emp m
ON e.mgr=m.empno;
```

Cross Join / Cartesian Join / Product Join:

- In Cross Join, Each record in one table will be joined with every record in another table.
- For CROSS JOIN we will not write any Join Condition.



GROUPA a

cid	cname
10	IND
20	AUS
30	WI

GROUPB b

cid	cname
40	PAK
50	ENG
60	NZ

ORACLE STYLE:

```
SELECT a.cname || ' VS ' || b.cname
FROM groupa a, groupb b;
```

ANSI STYLE:

```
SELECT a.cname || ' VS ' || b.cname
FROM groupa a CROSS JOIN groupb b;
```

Output:

```
IND VS PAK
IND VS ENG
IND VS NZ
AUS VS PAK
AUS VS ENG
AUS VS NZ
WI VS PAK
WI VS ENG
WI VS NZ
```

Joins	retrieve data from multiple tables
-------	------------------------------------

Equi	based on equality condition
------	-----------------------------

	matched
Left Outer	matched + unmatched from left
Right Outer	matched + unmatched from right
Full Outer	matched + unmatched from Left & Right
Self	A table is joined to itself
CROSS	each record in one table will be joined with every record in another

Sub Queries / Nested Queries:

- Writing query in another query is called "Sub Query / Nested Query".

Syntax to define Sub Query:

```
SELECT <column_list>  
FROM <table_name>  
WHERE <column> <operator> (<SELECT query>);
```

Outer / main / Parent query

Inner / Sub / Child query

- Inside query is called "Inner / Sub / Child Query"
- Outside query is called "Outer / Main / Parent Query".
- Normally, when WHERE condition value is unknown then find it by writing the Sub Query.
- Inner Query must be SELECT query only. It cannot be INSERT / DELETE / UPDATE query.
- Outer query can be SELECT / INSERT / UPDATE / DELETE query.
- Inner query must be written in parenthesis.
[(<subquery>)]
- Normally, First Inner query gets executed. Then Outer query gets executed.
- The result of Inner Query will become input for Outer Query.

Types of Sub Queries:

- Single Row Sub Query ✓
- Multi Row Sub Query
- Correlated Sub Query ✓
- Inline View / Inline Sub Query ✓
- Scalar Sub Query

Single Row Sub Query:

If sub query returns single row then it is called "Single Row Sub Query".

Examples on Single Row Sub Queries:

- Find max salary:

```
SELECT max(Sal) FROM emp;  
8712
```

- Display the emp name who is earning max salary:

Tables

VIEWS
INDEXES

PL/SQL
basics
control structures
procedures
functions

```
SELECT ename FROM emp
WHERE sal=(SELECT max(sal) FROM emp);
```

SMITH

Display the emp records who are earning more than
BLAKE:

BLAKE sal => 3762

```
SELECT * FROM emp
WHERE sal > (SELECT sal FROM emp
WHERE ename='BLAKE');
```

Find 2nd max salary:

2nd max sal => 6000

```
SELECT empno,ename,sal FROM emp
WHERE sal = 6000;
```

4000 where sal < max sal => sal<8000
3000
8000
6000
5000

4000
3000
8000
6000
5000

max sal
2nd max sal

```
SELECT max(sal) FROM emp
WHERE sal < (SELECT max(Sal) FROM emp);
```

Display the emp name who is earning 2nd max sal:

```
SELECT empno,ename,sal
FROM emp
WHERE sal = (SELECT max(Sal) FROM emp
WHERE sal<(SELECT max(Sal) FROM emp));
```

Find 3rd max salary:

WHERE sal < 2nd max sal

4000
3000
8000
6000
5000

Handwritten notes: 5000 is circled and labeled 'max sal'. 3000 is labeled '3rd max sal'.

```
SELECT max(sal) FROM emp
WHERE sal < (SELECT max(sal) FROM emp
WHERE sal < (SELECT max(sal) FROM emp));
```

Display the emp name who is earning 3rd max salary:

```
SELECT ename FROM emp
WHERE sal = 3rd max sal;
```

```
SELECT ename FROM emp
WHERE sal = (SELECT max(sal) FROM emp
WHERE sal < (SELECT max(sal) FROM emp
WHERE sal < (SELECT max(sal) FROM emp)));
```

Find most senior's hiredate:

hiredate	ASCENDING	DESCENDING
23-dec-1982	27-dec-1980	23-dec-1982
17-nov-1981	17-nov-1981	17-nov-1981
27-dec-1980	23-dec-1982	27-dec-1980

```
SELECT min(hiredate) FROM emp;
```

Display the most senior emp record:

```
SELECT empno,ename,hiredate FROM emp
WHERE hiredate = '17-DEC-1980';
```

Handwritten note: ↓ we have to find most senior's hiredate

```
SELECT empno,ename,hiredate FROM emp
WHERE hiredate = (SELECT min(hiredate)
```

FROM emp);

most junior's hiredate:

SELECT max(hiredate) FROM emp;

Display most junior's emp record:

**SELECT empno,ename,hiredate FROM emp
WHERE hiredate = (SELECT max(hiredate)
FROM emp);**

Multi Row Sub Query:

**If sub query returns multiple rows then it is called
"Multi Row Sub Query".**

Example:

**Display the emp records whose job titles are same as
ADAMS and BLAKE job titles:**

**ADAMS => CLERK
BLAKE => MANAGER**

display all managers & clerks records

**SELECT empno,ename,job,sal
FROM emp WHERE job IN(SELECT job FROM emp
WHERE ename IN('ADAMS','BLAKE'));**

Correlated Sub Query:

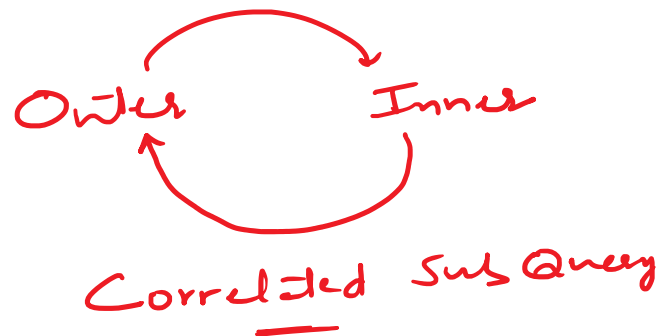
- **If outer query passes value to inner query then it is called "Correlated Sub Query".**
- **Normal Sub query, first inner query gets executed. then outer query gets executed. And Inner query gets executed only once.**
- **In Correlated Sub Query, First outer query gets executed. And Inner query gets executed for multiple times.**

Execution Order of Correlated Sub Query:

- **First Outer query gets executed. It selects a row & passes value to inner query.**
- **Inner query gets executed. Result will be given to outer query.**
- **Outer query condition will be tested. If condition is**

TRUE, it selects the row. Otherwise, row will not be selected.

- Above steps get executed repeatedly for every row selected by outer row



Display the emp records who are earning more than 30th dept's avrg sal:

```
SELECT empno,ename,sal FROM emp
WHERE sal > (SELECT avg(Sal) FROM emp
WHERE deptno=30);
```

Display the emp records who are earning more than their dept's average sal:

empno	ename	sal	deptno
1001	A	8000	20
1002	B	10000	20
1003	C	20000	30
1004	D	6000	30
1005	E	5000	10
1006	F	7000	10

deptno	avg(sal)
20	9000
30	13000
10	6000

```
SELECT empno,ename,sal,deptno FROM emp e
WHERE sal>(SELECT avg(Sal) FROM emp WHERE
deptno=e.deptno);
```

1002	B	10000	20
1003	C	20000	30
1006	F	7000	10

Find 3rd max salary:

empno	ename	sal	deptno
1001	A	4000	20
1002	B	10000	20
1003	C	20000	30
1004	D	8000	30
1005	E	5000	10

how many salaries > emp sal

no of sals> emp sal	0	max sal
no of sals>emp sal	1	2nd max
no of sals>emp sal	2	3rd max sal
no of sals>emp sal	3	4th max sal

```
SELECT sal FROM emp e
WHERE 2 = (SELECT count(DISTINCT sal)
FROM emp WHERE sal>e.sal);
```

8000

Find 4th max sal:

```
SELECT sal FROM emp e
WHERE 3 = (SELECT
count(DISTINCT sal) FROM emp
WHERE sal>e.sal);
```

Find 5th max sal:

```
SELECT sal FROM emp e
WHERE 4 = (SELECT
count(DISTINCT sal) FROM emp
WHERE sal>e.sal);
```

Find nth max sal:

```
SELECT DISTINCT sal FROM emp e
WHERE &n-1 = (SELECT
count(DISTINCT sal) FROM emp
WHERE sal>e.sal);
```

Sub Query:

Writing query in another query

```
SELECT ..... (SELECT ....);
```

Types of sub queries:

Single row sub query	sub query returns 1 row
multi row sub query	sub query returns multiple rows
correlated sub query	outer query passes value

	to inner query
--	----------------

- inline view
- scalar sub query

Inline View / Inline Sub Query:

- If sub query is written in **FROM** clause then it is called "Inline View".
- To control the execution order of clauses we use it.
- This sub query acts like table

```
SELECT empno,ename,sal  
FROM emp;
```

Execution Order:

```
FROM  
WHERE  
GROUP BY  
HAVING  
SELECT  
DISTINCT  
ORDER BY
```

Examples on Inline View:

Display the emp records whose annual salary is more than 30000:

```
SELECT empno,ename,sal,  
sal*12 as an_sal FROM emp  
WHERE an_sal>30000;
```

ERROR: an_Sal invalid identifier

We cannot use alias name in WHERE clause. Because, WHERE gets executed before SELECT

```
SELECT * FROM (SELECT empno,ename,sal,  
sal*12 as an_sal FROM emp)  
WHERE an_sal>30000;
```

Find top 3 salaries:

```
SELECT * FROM (SELECT empno,ename,sal,  
dense_rank() over(order by sal desc) as rank  
FROM emp)  
WHERE rank<=3;
```

Find top n salaries:

```
SELECT * FROM (SELECT empno,ename,sal,
dense_rank() over(order by sal desc) as rank
FROM emp)
WHERE rank<=&n;
```

Find 3rd max sal:

```
SELECT * FROM (SELECT empno,ename,sal,
dense_rank() over(order by sal desc) as rank
FROM emp)
WHERE rank=3;
```

Find nth max sal:

```
SELECT * FROM (SELECT empno,ename,sal,
dense_rank() over(order by sal desc) as rank
FROM emp)
WHERE rank=&n;
```

Scalar Sub Query:

- If sub query is written in **SELECT** clause then it is called "Scalar Sub Query".

Example:

Find no of records in emp & dept tables:

dept	emp
-----	-----
4	14

```
SELECT count(*) FROM dept; -- 4
```

```
SELECT count(*) FROM emp; --14
```

```
SELECT (SELECT count(*) FROM dept) as dept,
(SELECT count(*) FROM emp) as emp
FROM dual;
```

dept

4

Calculate share of each dept in salaries:

```
SELECT deptno, sum(Sal) as dept_sum_Sal,
(SELECT sum(Sal) FROM emp) as tot_Sum_Sal,
Trunc((sum(Sal)/(SELECT sum(Sal) FROM emp))*100,2) as
perctg
```

FROM emp
GROUP BY deptno
ORDER BY deptno;

SQL

5 sub languages

DDL => CREATE, ALTER, DROP, TRUNCATE, RENAME, FLASHBACK, PURGE

DRL => SELECT

DML => INSERT, UPDATE, DELETE, INSERT ALL, MERGE

DCL => GRANT, REVOKE

TCL => ROLLBACK (Cancel), COMMIT (save), SAVEPOINT

Built-In Functions

Text, Math, conversion, date, aggregate, miscellaneous

Clauses

SELECT	specify column list
FROM	specify table list
WHERE	write conditions on rows
GROUP BY	to group the records according to specific column
HAVING	to write conditions on groups
DISTINCT	to avoid duplicate values
ORDER BY	to arrange records in ascending or descending order

Joins:

to retrieve data from multiple tables

Types:

Equi	based on equality condition matched records only
Left Outer	matched + unmatched from left
Right Outer	matched + unmatched from right
Full Outer	matched + unmatched from left & right
Self Join	If a table is joined to itself one table record will be joined with another record in same table
Cross Join	Each record in 1 table joined with every record in another table
Non-Equi	based on other than equality condition

Sub Queries:

writing query in another query

Types of Sub queries:

Single row sub query	sub query returns 1 row
Multi row sub query	sub query returns multiple rows

Correlated Sub Query	outer query passes value to inner query
Inline view	if sub query written in FROM clause
Scalar Sub query	if sub query is written SELECT clause

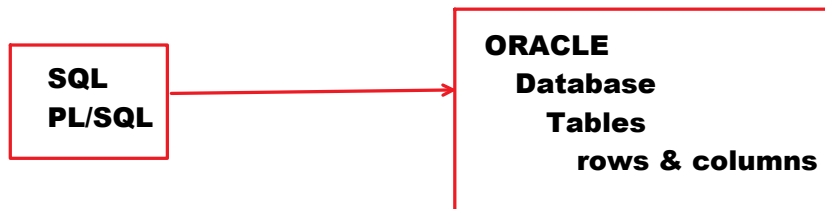
SQL

PL/SQL

Tables

Views

Indexes



**SQL & PL/SQL are the languages,
those are used to communicate
with the ORACLE database.**

PL/SQL:

- PL => Procedural Language
- SQL => Structured Query Language.
- PL/SQL is procedural language. We can write a set of statements or programs here.
- PL/SQL is extension of SQL. All SQL queries we can write as statements in PL/SQL.

Advantages:

- improves the performance
- provides condition control structures
- provides looping control structure
- provides modularity
- provides exception handling
- provides security

SQL:

Non-Procedural

we will not write set of stmts or programs just we write queries

Function

--set of statements

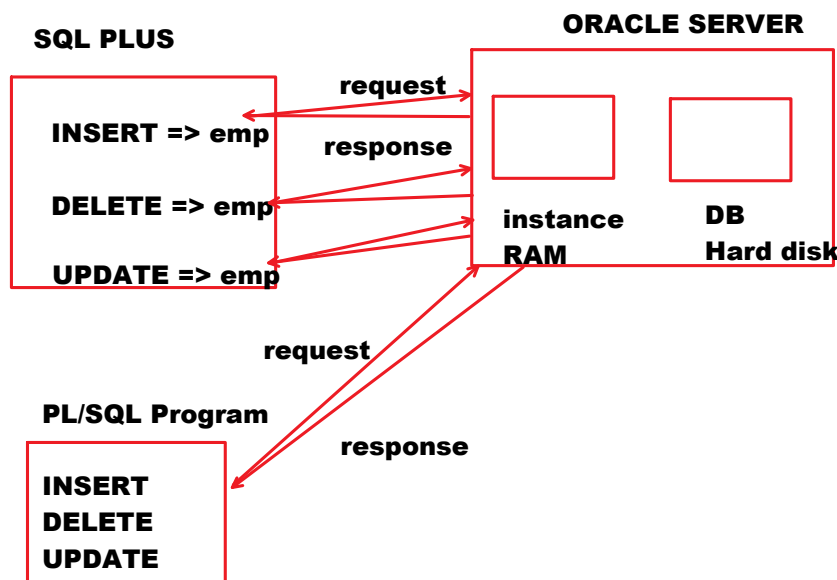
Method

-- set of statements

PROCEDURE

--set of statements

improves the performance:



PL/SQL program reduces number of requests to the Database Server. So, performance will be improved.

provides condition control structures:

- Based on conditions we can perform the actions by using conditional control structures.
- PL/SQL provides conditional control structures `if..then`, `if..then..else`, `if..then..elsif`

provides Looping control structures:

- we can perform the actions repeatedly using loops.
- PL/SQL provides loops like Simple Loop, while

loop & for loop.

provides modularity:

- **modularity => is a programming style**
- **dividing large program into small parts. Every small part is called Procedure (or) Function**
- **Advantages:**
 - **Provides Reusability.**
 - **decreases length of code.**
 - **improves the understandability**
 - **better maintenance**

provides Exception Handling:

- **Exception => run-time error**
- **Exception Handling => the mechanism of handling run-time errors**
- **When run-time error occurs our application will be closed in the middle of execution. that's why we must handle it.**

Smart Phone => 32 GB => 32 GB memory is full
Camera App => selfie

Provides Security:

- **All procedures & functions are stored in centralized location. Only authorized users can use them.**

Types of Blocks:

There are 2 types of blocks in PL/SQL. They are:

- **Anonymous Block => a block without name**
- **Named Block => a block with name**

Exs: function, procedure, trigger, package

Syntax of Anonymous Block:

<pre>DECLARE --declare the variables BEGIN --executable statements END; /</pre>	} → } →	DECLARATION PART (OPTIONAL) EXECUTION PART
---	---	--

In C:

printf(): => built-in function
used to print the data on output screen
defined in "stdio.h" header file

In PL/SQL:

put_line():

- **"put_line()" is a built-in procedure that is defined in "dbms_output" package.**
- **it is used to print the data on screen.**

Syntax:

dbms_output.put_line(<string>);

Ex:

dbms_output.put_line('hello');

Program to print "hello" on output screen:

BEGIN

dbms_output.put_line('hello');

END;

/

In any text editor type above program. Save it in
"D:" Drive "Oracle4pm" folder with name
"HelloDemo.sql"

Ex:

Notepad

Edit Plus

Notepad++

Open SQL PLUS:

Syntax to compile & run PL/SQL program:
SQL> @<path>

SQL> SET SERVEROUTPUT ON

SQL> @D:\oracle4pm\HelloDemo.sql

Output:

hello

SET SERVEROUTPUT ON:

If SERVEROUTPUT is OFF, messages cannot be
sent to output screen. To send messages to
output, we must set SERVEROUTPUT as ON

Data Types in PL/SQL:

Integer Related Data types:

Number(p)

integer

int

binary_integer => PL/SQL only

pls_integer => PL/SQL only

Floating Point Data Types:

Number(p,s)

float

binary_float

binary_double

Character Related:

char(n)

varchar2(n)

long

CLOB

string(n) => PL/SQL only

nchar(n)

nvarchar2(n)

nCLOB

Date & Time Related:

Date

Timestamp

Binary Related:

BFILE

BLOB

Attribute related data types:

%type => pl/sql only

%rowtype => pl/sql only

Declaring variable:

Syntax:

<variable> <data_type>;

In C:

int x;


```
<variable> <data_type>;
```

EX:

```
x number(4);  
y varchar2(10);  
z date;
```

```
int x;
```

Assigning values:

```
:= Assignment Operator
```

```
x=25; //25 constant  
y=x;  //x variable  
z=x+y; //x+y expression
```

Syntax:

```
<variable> := <constant / variable / expression>;
```

Ex:

```
x := 25;  
y := x;  
z := x+y;
```

Program to add 2 numbers:

DECLARE

```
x number(4);  
y number(4);  
z number(4);
```

BEGIN

```
x := 20;  
y := 30;
```

```
z := x+y;
```

```
dbms_output.put_line('sum=' || z);  
dbms_output.put_line('sum of ' || x || ' and ' || y || ' is ' || z);
```

END;

/

Reading values at run time:

student

```
sid sname
```

```
INSERT INTO student VALUES(&sid,&sname);
```

enter value for sid:
enter value for sname:

Read x value:

x := &sid;

enter value for x: 25

**Program to multiply 2 numbers. Read 2 nums
at run time:**

Output:

**enter value for x:2
enter value for y:3
product=6**

Output:

**enter value for x:10
enter value for y:5
product=50**

/*	*/	multi line comment
--		single line comment

DECLARE

x number(4);

y number(4);

BEGIN

x := &x; --reading x

y := &y; --reading y

dbms_output.put_line('product=' || (x*y));

END;

/

enter value for x: 10

old : x := &x

new: x := 10

enter value for y: 5

old : y := &y

new: y := 5

product=50

**to avoid of displaying old & new parameters
write as following:**

SQL> SET VERIFY OFF

SQL>@d:\oracle4pm\productdemo

enter value for x: 2

enter value for y: 3

product=6

Program to print student details:

name: Ramu	→	name
section: A	→	sec
dob: 25-dec-1998	→	dob
marks: 786	→	m
avrg marks: 78.6	→	a

```

DECLARE
    name varchar2(10);
    sec CHAR;
    m number(4);
    a number(5,2);
    dob date;
BEGIN
    name := '&name';
    sec := '&section';
    dob := '&dob';
    m := &m;
    a := &a;

    dbms_output.put_line(name || ' ' || sec || ' ' || dob
    || ' ' || m || ' ' || a);

END;
/

```

Writing SQL commands in PL/SQL:

- DML, DDL & TCL commands can be used directly in PL/SQL program.
- We cannot use DDL & DCL commands in PL/SQL program directly. But, using "Dynamic SQL" we can use DDL & DCL commands also in PL/SQL program.

SELECT command syntax in PL/SQL:

```

SELECT <column_list> INTO <variable_list>
FROM <table_name>
WHERE <condition>;

```

Ex:

```

SELECT ename,sal INTO x,y
FROM emp
WHERE empno=7369;

```

Handwritten boxes showing the result of the SELECT statement:

SMITH

5000

empno	ename	sal
7369	SMITH	5000

Program to display the emp record of given empno:

enter value for empno:7369
SMITH 8000

enter value for empno:7566
JONES 3900

Program:

emp			
empno	ename	job	sal

DECLARE

v_empno Number(4);
v_ename varchar2(3);
v_sal number(7,2);

BEGIN

v_empno := &empno;

SELECT ename,sal **INTO** v_ename,v_sal
FROM emp **WHERE** empno=v_empno;

dbms_output.put_line(v_ename || ' ' || v_sal);

END;

/

%type:

- is attribute related data type
- is used to declare a variable with table column's data type & field size.
- It avoids mismatch between variable type, size & table column's type,size.

Syntax:

<variable_name> <table_name>.<field_name>%type

Ex:

v_ename emp.ename%type;

emp table's ename column's data type will be taken for "v_ename" variable.

v_sal emp.sal%type;

emp table's sal column's data type will be taken for "v_sal" variable.

Display the emp record of given empno:

DECLARE

v_empno emp.empno%type;
v_ename emp.ename%type;

emp	
empno	number(4)

```

v_sal emp.sal%type;
BEGIN
  v_empno := &empno;

  SELECT ename,sal INTO v_ename,v_sal
  FROM emp WHERE empno=v_empno;

  dbms_output.put_line(v_ename || ' ' || v_sal);

END;
/

```

Program to update the salary of given empno with given amount:

```

enter ... empno: 7369
enter ... amount: 2000

```

empno	ename	sal
7369	SMITH	8000

```

DECLARE
  v_empno emp.empno%type;
  v_amount Number(7,2);
BEGIN
  v_empno := &empno;
  v_amount := &amount;

  UPDATE emp SET sal=sal+v_amount
  WHERE empno=v_empno;
  COMMIT;

  dbms_output.put_line('salary increased...!');

```

emp

empno	ename	sal
7369	SMITH	8000

```

END;
/

```

Program to delete an emp record of given empno:

```

enter ... empno: 7934
record deleted...!

```

```

DECLARE
  v_empno emp.empno%type;
BEGIN
  v_empno := &empno;

  DELETE FROM emp WHERE empno=v_empno;
  COMMIT;

```

```
        dbms_output.put_line('record deleted..!');  
    END;  
/
```

Program to insert student record into table:

STUDENT

sid	sname	m1
-----	-------	----

```
CREATE TABLE student  
(  
    sid Number(4),  
    sname Varchar2(10),  
    m1 Number(3)  
);
```

DECLARE

```
    v_sid student.sid%type;  
    v_sname student.sname%type;  
    v_m1 student.m1%type;
```

BEGIN

```
    v_sid := &sid;  
    v_sname := '&sname';  
    v_m1 := &m1;
```

```
    INSERT INTO student VALUES(v_sid,v_sname,v_m1);  
    COMMIT;
```

```
        dbms_output.put_line('record inserted..!');  
    END;  
/
```

Control Structures

Wednesday, August 17, 2022 4:25 PM

BEGIN

dbms_output.put_line('hi');

dbms_output.put_line('welcome');

dbms_output.put_line('bye');

END;

/

*Sequentially
Statements*

hi
welcome
bye

*Control
Structure*

DECLARE

x INT := 5;

BEGIN

IF x>5 **THEN**

dbms_output.put_line('hi');

END IF;

IF x<5 **THEN**

dbms_output.put_line('welcome');

END IF;

IF x=5 **THEN**

dbms_output.put_line('bye');

END IF;

END;

/

5

*Control
Statements*

bye

- Normally, PL/SQL program gets executed sequentially. To change the sequential execution we use Control Structures.
- Control Structures are used to control the flow of execution of statements.

PL/SQL provides following control structures:

Conditional	IF .. THEN IF ..THEN .. ELSE IF ..THEN .. ELSIF Nested If CASE
Looping	Simple Loop For Loop While Loop
Jumping	EXIT EXIT WHEN GOTO CONTINUE

Conditional Control Structures:

Conditional Control Structures execute the statements based on conditions.

IF .. THEN:

Syntax:

```
IF <condition> THEN  
  --Statements  
END IF;
```

Program to check whether the person is eligible to vote or not:

```
DECLARE  
  age INT;  
BEGIN  
  age := &age;  
  
  IF age>=18 THEN  
    dbms_output.put_line('Eligible to vote');  
  END IF;  
  
END;  
/
```

**Program to delete an emp record of given empno.
If experience is more than 40 years then only
delete the record:**

enter value for empno: 7900
experience: 39

enter value for empno: 7521
experience: 41
record deleted..!

```
DECLARE  
  v_empno emp.empno%type;
```



```

v_hiredate date;
v_exp INT;
BEGIN
v_empno := &empno;

SELECT hiredate INTO v_hiredate FROM emp
WHERE empno=v_empno;

v_exp := (sysdate-v_hiredate)/365;

dbms_output.put_line('experience=' || v_exp);

IF v_exp>40 THEN
DELETE FROM emp WHERE empno=v_empno;
COMMIT;
dbms_output.put_line('record deleted..!');
END IF;

END;
/

```

IF .. THEN .. ELSE:

Syntax:

```

IF <condition> THEN
--statements
ELSE
--statements
END IF;

```

Program to check whether the given number is even or odd:

Even => 2,4,6,8, => divide with 2 => remainder 0

Odd => 1,3,5,7, => divide with 2 => remainder 1

Handwritten examples of division by 2:

- Even numbers (remainder 0):
 - $2 \overline{) 12} \begin{matrix} 6 \\ 12 \\ \hline 0 \end{matrix}$
 - $2 \overline{) 18} \begin{matrix} 9 \\ 18 \\ \hline 0 \end{matrix}$
 - $2 \overline{) 22} \begin{matrix} 11 \\ 22 \\ \hline 0 \end{matrix}$
- Odd numbers (remainder 1):
 - $2 \overline{) 13} \begin{matrix} 6 \\ 12 \\ \hline 1 \end{matrix}$
 - $2 \overline{) 15} \begin{matrix} 7 \\ 14 \\ \hline 1 \end{matrix}$
 - $2 \overline{) 9} \begin{matrix} 4 \\ 8 \\ \hline 1 \end{matrix}$

A wavy line separates the even and odd examples.

```

DECLARE
  n INT;
BEGIN
  n := &n;

  IF mod(n,2)=0 THEN
    dbms_output.put_line('EVEN');
  ELSE
    dbms_output.put_line('ODD');
  END IF;

END;
/

```

Program to increase salary of given empno as following:
if experience is 40 or more => increase 10% on sal
if experience is <40 => increase 20% on sal

```

DECLARE
  v_empno emp.empno%type;
  v_hiredate date;
  v_exp INT;
  v_per INT;
BEGIN
  v_empno := &empno;

  SELECT hiredate INTO v_hiredate FROM emp
  WHERE empno=v_empno;

  v_exp := (sysdate-v_hiredate)/365;

  IF v_exp>=40 THEN
    v_per := 10;
  ELSE
    v_per := 20;
  END IF;

  UPDATE emp SET sal=sal+sal*v_per/100
  WHERE empno=v_empno;

  COMMIT;
  dbms_output.put_line('experience=' || v_exp);
  dbms_output.put_line(v_per || '% salary increased..!');
END;
/

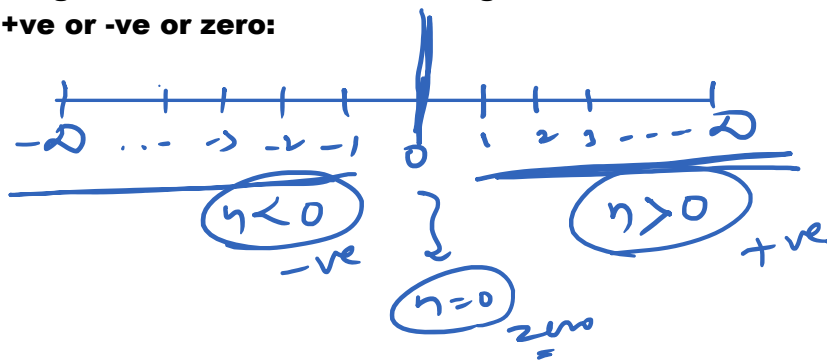
```

IF .. THEN .. ELSIF:

Syntax:

```
IF <condition-1> THEN
  --statements
ELSIF <condition-2> THEN
  --statements
ELSIF <condition-3> THEN
  --statements
.
.
ELSE
  --statements
END IF;
```

Program to check whether the given number is +ve or -ve or zero:



```
DECLARE
  n INT;
BEGIN
  n := &n;

  IF n>0 THEN
    dbms_output.put_line('+ve');
  ELSIF n<0 THEN
    dbms_output.put_line('-ve');
  ELSE
    dbms_output.put_line('zero');
  END IF;

END;
```

/

Program to increase salary of given empno as following:

if job is manager => increase 10%

if job is clerk => increase 20%

others => increase 15%

read empno

retrieve job of given empno & hold in variable

DECLARE

v_empno emp.empno%type;

v_job emp.job%type;

v_per INT;

BEGIN

v_empno := &empno;

SELECT job INTO v_job FROM emp

WHERE empno=v_empno;

IF v_job='MANAGER' THEN

v_per := 10;

ELSIF v_job='CLERK' THEN

v_per := 20;

ELSE

v_per := 15;

END IF;

UPDATE emp SET sal=sal+sal*v_per/100

WHERE empno=v_empno;

COMMIT;

dbms_output.put_line(v_per || '% salary incremented..!');

END;

/

Nested If:

Writing If..Then in another If..Then is called "Nested If".

Syntax:

outer if

```
IF <condition-1> THEN
  inner if
  IF <condition-2> THEN
    --statements
  END IF;
END IF;
```

Max marks: 100

Min marks: 40 marks for pass in each sub

Program to find total, average & result of given sid:

if result is pass then only calculate average.

if average is 60 or more => First Division

if average is b/w 50 to 59 => Second Division

if average is b/w 40 to 49 => Third Division

STUDENT

CREATE TABLE student

```
(  
sid number(4),  
sname varchar2(10),  
m1 number(3),  
m2 number(3),  
m3 number(3)  
);
```

INSERT INTO student values(1001,'A',60,80,50);

INSERT INTO student values(1002,'B',80,30,70);

COMMIT;

DECLARE

v_sid student.sid%type;

x student.m1%type;

y student.m2%type;

z student.m3%type;

total INT;

avrg INT;

BEGIN

v_sid := &sid;

SELECT m1,m2,m3 INTO x,y,z FROM student

WHERE sid=v_sid;

IF x>=40 AND y>=40 AND z>=40 THEN

total := x+y+z;

avrg := total/3;

dbms_output.put_line('total=' || total);

dbms_output.put_line('avrg=' || avrg);

IF avrg>=60 THEN

dbms_output.put_line('FIRST DIVISION');

ELSIF avrg>=50 THEN

sid	sname	M1	M2	M3
1001	A	60	80	50
1002	B	80	30	70

```

        dbms_output.put_line('SECOND DIVISION');
    ELSE
        dbms_output.put_line('THIRD DIVISION');
    END IF;
ELSE
    dbms_output.put_line('FAIL');
END IF;

END;
/

```

CASE:

It can be used in 2 styles:

• Simple CASE	can check equality condition only
• Searched CASE	can check any condition

Simple CASE:

Syntax:

```

CASE <expression>
WHEN <value-1> THEN
    --statements
WHEN <value-2> THEN
    --statements
.
.
ELSE
    --statements
END CASE;

```

Program to check whether the given number is even or odd using SIMPLE CASE:

even => 2,4,6,8, ... => divide with 2 => remainder 0

odd => 1,3,5,7, ... => divide with 2 => remainder 1

```

DECLARE
    n INT;
BEGIN
    n := &n;

```

```

CASE mod(n,2)
WHEN 0 THEN
    dbms_output.put_line('EVEN');
WHEN 1 THEN
    dbms_output.put_line('ODD');
END CASE;

END;
/

```

Assignment:

Increase salary of given empno as following:

deptno = 10 => increase 10%

deptno = 20 => increase 20%

others => increase 15%

Searched Case:

Syntax:

```

CASE
WHEN <condition-1> THEN
    --statements
WHEN <condition-2> THEN
    --statements
.
.
ELSE
    --statements
END CASE;

```

**Program to check whether the given number is
+ve or -ve or zero:**

```

DECLARE
    n INT;
BEGIN
    n := &n;

```

```

CASE
WHEN n>0 THEN
    dbms_output.put_line('+ve');
WHEN n<0 THEN
    dbms_output.put_line('-ve');
ELSE
    dbms_output.put_line('zero');
END CASE;
END;
/

```

Looping Control Structures:

are used to execute the statements repeatedly.

PL/SQL provides 3 looping control structures:

- Simple Loop
- While Loop
- For Loop

Simple Loop:

Syntax:

```

LOOP

    --statements

END LOOP;

```

Program to print numbers from 1 to 4:

i 1 2 3 4	<pre> DECLARE i INT := 1; n INT; BEGIN n := &n; LOOP dbms_output.put_line(i); EXIT WHEN i=n; i:=i+1; </pre>
---	--


```

        END LOOP;
    END;
/

```

Looping Control Structures:

Simple Loop

While Loop

For Loop

i

1
2
3
4

```

DECLARE
    i INT := 1;
BEGIN
    LOOP
        dbms_output.put_line(i);
        EXIT WHEN i=4;
        i := i+1;
    END LOOP;
END;
/

```

```

DECLARE
    i INT := 1;
BEGIN
    LOOP
        dbms_output.put_line(i);
        IF i=4 THEN
            EXIT;
        END IF;
        i := i+1;
    END LOOP;
END;
/

```

IF i=4 THEN EXIT; END IF;

=

EXIT WHEN i=4;

NOTE:

to terminate SIMPLE LOOP use EXIT or EXIT WHEN

While Loop:

Syntax:

```

While <condition>
LOOP
  --Statements
END LOOP;

```

The statements in while get executed as long as the condition is true. When the condition is false, it terminates the loop.

Program to print numbers from 1 to 4 using while loop:

i 1 2 3 4	<pre> DECLARE i INT := 1; BEGIN WHILE i<=4 LOOP dbms_output.put_line(i); i := i+1; END LOOP; END; / </pre>
---	---

For Loop:

Syntax:

```

FOR <variable> IN [REVERSE] <lower> .. <upper>
LOOP
  --Statements
END LOOP;

```

Program to print numbers from 1 to 4 using for loop:

i 1 2	<pre> BEGIN FOR i IN 1 .. 4 LOOP </pre>
---	--

1
2
3
4

```
BEGIN
  FOR i IN 1 .. 4
  LOOP
    dbms_output.put_line(i);
  END LOOP;
END;
/
```

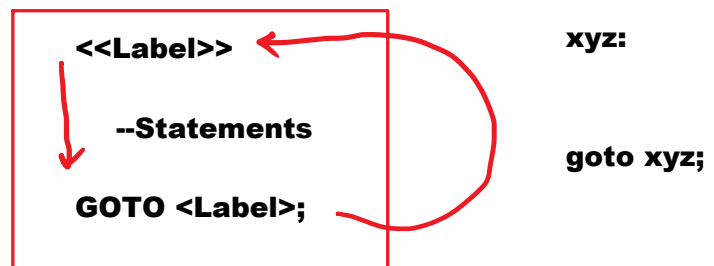
NOTE:

- We have no need to declare Loop Variable. Loop variable will be declared implicitly as number type.
- Loop variable scope is limited to that "For Loop" only. We cannot use out of the Loop.
- Always Loop variable increment by value is 1.

GOTO:

- is used to transfer the control to specified label.

Syntax:



i

1
2
3
4

```
DECLARE
  i INT := 1;
BEGIN
  <<nareshit>>
    dbms_output.put_line(i);
    i := i+1;
    IF i<=4 THEN
      GOTO nareshit;
    END IF;
END;
/
```

Continue:

- used to skip the current iteration & continue the next iteration
- It can be used in Loop only.

Example:

Program to print numbers from 1 to 10 except 7:

i

**1
2
3
4
6
8
9
10**

```
BEGIN  
  for i IN 1 .. 10  
  LOOP  
    IF i=5 OR i=7 THEN  
      continue;  
    END IF;  
    dbms_output.put_line(i);  
  END LOOP;  
END;  
/
```

Procedures

Friday, August 19, 2022 5:03 PM

Procedure:

- **Procedure is a named block of statements that gets executed on calling.**

Types of procedures:

2 types:

- **Stored Procedure**
- **Packaged Procedure**

Stored Procedure / Stand Alone Procedure:

If procedure is defined directly in schema [user] then it is called "Stored Procedure".

Ex:

```
c##batch4pm SCHEMA[USER]
PROCEDURE withdraw => Stored Procedure
```

Packaged Procedure:

If procedure is defined inside of package then it is called "Packaged Procedure".

Ex:

```
c##batch4pm SCHEMA
PACKAGE bank
PROCEDURE withdraw => Packaged procedure
```

Syntax to define the Stored Procedure:

```
CREATE OR REPLACE PROCEDURE
<procedure_name>([<argument_list>])
IS / AS
  --Declare the variables
BEGIN
  --Executable Statements
END;
/
```

C-Language:

Function

-- set of statements

Java:

Method

--set of statements

PL/SQL:

procedure:

is a set of statements

function:

is a set of statements

Define a procedure to add 3 numbers:

```
addition(10,20,30)
10+20+30 = 60
```

```
CREATE OR REPLACE PROCEDURE  
addition(x Number,y Number,z Number)  
IS  
    r Number(4);  
BEGIN  
    r := x+y+z;  
    dbms_output.put_line('sum=' || r);  
END;  
  
/
```

```
addition(int x,int y,int z)
{
}
main()
{
    addition(10,20,30);
}
```

Procedure can be called in 3 ways:

- **From SQL prompt**
- **From another PL/SQL program [main program]**
- **From Front-End Application [Java, Python, C#]**

From SQL prompt:

"Exec / Execute" command is used to call a procedure from SQL prompt.

```
SQL> exec addition(10,20,30);
sum=60
```

Advantages:

- **Procedure improves the performance.**
- **provides Reusability.**
- **decrease length of code.**

- **From another PL/SQL program [main program]:**

--main program:

DECLARE

a INT;

b INT;

```
c INT;
```

BEGIN

a := &a;

b := &b;

```
c := &c;
```

addition(a,b,c); --calling from PL/SQL program

END;
/

Types of parameters [arguments]:

Parameter [Argument]:

- **Parameter is a local variable which is declared in procedure header.**

Syntax to declare parameter:

<parameter_name> <parameter_type> <data_type>

Ex:

x IN Number

y OUT Number

z IN OUT Number

There are 3 types of parameters:

- **IN [Default]**
- **OUT**
- **IN OUT**

IN:

- **It is default one.**
- **It is used to bring value into procedure from out of procedure.**
- **In procedure call it can be variable or constant.**
- **It is read-only variable.**

CREATE OR REPLACE PROCEDURE

addition(x IN Number,y IN Number,z OUT Number)

addition(5,4,a) --procedure call

p:=10

q:=20

addition(p,q,r) --procedure call

OUT:

- **It is used to send the value out of the procedure.**
- **In procedure call, it must be variable only.**

IN OUT:

- **Same variable can be used to bring value into procedure & send value out of the procedure.**

- In procedure call, it must be variable only.

Define a procedure to add 2 numbers. Send the result out of the procedure using OUT parameter:

```
CREATE OR REPLACE PROCEDURE
addition(x IN Number, y IN Number, z OUT Number)
IS
BEGIN
    z := x+y;
END;
/
```

```
SQL> variable a number
SQL> exec addition(5,4,:a);
SQL> PRINT a
Output:
```

```
-----
a
----
9
```

Bind Variable:

- A variable which is declared at SQL prompt is called "bind Variable".
- TO access the value we use bind operator :

Caling from another PL/SQL program:

```
DECLARE
    a INT;
    b INT;
    c INT;
BEGIN
    a := &a;
    b := &b;

    addition(a,b,c); --procedure call

    dbms_output.put_line('sum=' || c);
END;
/
```


Parameter mapping Techniques:

There are 3 parameter mapping techniques:

- Positional Notation
- Named Notation
- Mixed Notation

CREATE OR REPLACE PROCEDURE

```
addition(x Number, y Number, z Number)
IS
  r INT;
BEGIN
  r := x+y+z;
  dbms_output.put_line('x=' || x || ' ' || 'y=' || y || ' ' || 'z=' || z);
  dbms_output.put_line('sum=' || r);
END;
/
```

Position Notation:

In this, based on positions of parameters values will be mapped.

Ex:

```
addition(x Number, y Number, z Number)
addition(10,20,30)
```



Named Notation:

based on the names of parameters values will be mapped.

Ex:

```
addition(x Number, y Number, z Number)
addition(z=>10,x=>20,y=>30)
```



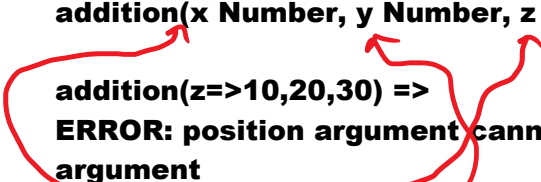
Mixed Notation:

based on position of parameters & names of parameters values are mapped.

Ex:

```
addition(x Number, y Number, z Number)
```

```
addition(z=>10,20,30) =>
```



ERROR: position argument cannot be followed by named argument

ERROR: position argument cannot be followed by named argument

addition(10,z=>20,y=>30)

Define a procedure to update salary of an empno with specific amount. Send the updated salary out of the procedure using OUT parameter:

CREATE OR REPLACE PROCEDURE

**update_salary(p_empno IN emp.empno%type,
p_amount IN FLOAT, p_sal OUT emp.sal%type)**

IS

BEGIN

**UPDATE emp SET sal=sal+p_amount
WHERE empno=p_empno;**

**SELECT sal INTO p_sal FROM emp
WHERE empno=p_empno;**

END;

/

SQL> variable s Number

SQL>update_salary(7369,2000,:s)

SQL> print s

ACCOUNT

AcNo	Name	Balance
1001	A	200000
1002	B	500000

Define a procedure to perform withdraw operation:

CREATE TABLE account

(

acno number(4),

Name varchar2(10),

balance number(9,2)

);

INSERT INTO account VALUES(1001,'A',200000);

```
INSERT INTO account VALUES(1002,'B',500000);
COMMIT;
```

CREATE OR REPLACE PROCEDURE

```
withdraw(p_acno account.acno%type,
p_amount FLOAT)
```

```
IS
```

```
    v_balance FLOAT;
```

```
BEGIN
```

```
    SELECT balance INTO v_balance FROM account
    WHERE acno=p_acno;
```

```
    IF p_amount>v_balance THEN
```

```
        dbms_output.put_line('Insufficient Balance..!');
```

```
    ELSE
```

```
        UPDATE account SET balance=balance-p_amount
```

```
        WHERE acno=p_acno;
```

```
        COMMIT;
```

```
        dbms_output.put_line('amount debited...!');
```

```
    END IF;
```

```
END;
```

```
/
```

```
SQL> exec withdraw(1001,300000);
```

ACCOUNT

AcNo	Name	Balance
1001	A	200000
1002	B	500000

Define a procedure to perform deposit operation.

Updated balance send out of the procedure:

CREATE OR REPLACE PROCEDURE

```
deposit(p_acno IN account.acno%type,
```

```
p_amount IN FLOAT,
```

```
p_balance OUT account.balance%type)
```

```
IS
```

```
BEGIN
```

```
    UPDATE account SET balance=balance+p_amount
```

```
    WHERE acno=p_acno;
```

```
    COMMIT;
```

```
    dbms_output.put_line('amount credited...!');
```

```
    SELECT balance INTO p_balance FROM account
```

```
    WHERE acno=p_acno;
```

```
END;
```

```
/
```

variable b number

```
exec deposit(1002,100000,:b);
```

ACCOUNT

AcNo	Name	Balance
1001	A	200000
1002	B	500000

amount debited...!

print b

PROCEDURE:

is a named block of statements that gets executed on calling

2 types:

stored procedure

packaged procedure

Parameters:

3 types:

IN	bringing value into procedure
OUT	sending value out of procedure
IN OUT	used to bring value into procedure & same variable can be used to send the result out of procedure

Function:

- **Function is a named block of statements that gets executed on calling.**
- **To perform DML operations define PROCEDURE WHERE AS to perform fetch operations [SELECT] or calculations define FUNCTION.**

deposit => update => procedure
withdraw => update => procedure
check_balance => select => function
trans_statement => select => function
experience => calculation => function

Syntax:

```
CREATE OR REPLACE FUNCTION <function_name>  
[(<argument_list>)] RETURN <type>  
IS  
BEGIN  
    //Statements  
END;  
/
```

Define a function to add 2 numbers:

```
CREATE OR REPLACE FUNCTION  
addn(x number,y number) RETURN number  
IS
```

```
BEGIN  
    RETURN x+y;  
END;  
/
```

A function can be called using 3 ways:

- **From SQL prompt**
- **From another PL/SQL program [main program]**
- **From Front-End Application [Java, Python, C#]**

From SQL prompt:

```
SQL> SELECT addition(2,3) FROM dual;
```

From another PL/SQL program [main program]

```
DECLARE  
    a INT;  
    b INT;  
    c INT;  
BEGIN  
    a := &a;  
    b := &b;  
  
    c := addn(a,b);  --function call  
  
    dbms_output.put_line('sum=' || c);  
END;  
/
```

Define a function to check the balance:

CREATE OR REPLACE FUNCTION

```

check_balance(p_acno account.acno%type)
RETURN Number
IS
    v_balance account.balance%type;
BEGIN
    SELECT balance INTO v_balance FROM account
    WHERE acno=p_acno;

    RETURN v_balance;
END;
/

```

Define a function to Calculate experience of an employee:

```

CREATE OR REPLACE FUNCTION
experience(p_empno emp.empno%type)
RETURN Number
IS
    v_hiredate DATE;
BEGIN
    SELECT hiredate INTO v_hiredate FROM emp
    WHERE empno=p_empno;

    RETURN trunc((sysdate-v_hiredate)/365);
END;
/

```

```

SELECT experience(7369) FROM dual;

```

Differences b/w Procedure & Function:

Procedure	Function
• to perform DML operations define it • cannot return the value. • Using OUT parameter we can send value of procedure • EXEC command is used to call it from SQL prompt	• to perform fetch operations or calculations define it • function returns the value. return is mandatory • RETURN is used to send value out of function • SELECT command is used to call it from SQL prompt

Views:

- **View is a Database Object.**
- **View is a virtual table.**
- **Virtual Table means, it does not contain physical data. It does not occupy the memory.**
- **A view holds SELECT query.**
- **A view will be created on table. A table on which view is created is called "Base Table".**
- **When we retrieve data through view, it retrieves data from base table only.**

Advantages:

- **provides security for the data.**
- **simplifies the queries.**

Syntax to create the view:

```
CREATE VIEW <view_name>  
AS  
<SELECT query>;
```

Example:

Log in as DBA:

GRANT create view TO c##batch4pm;

username: c##batch4pm

CREATE VIEW v1

AS

SELECT empno,ename,job FROM emp;

v1

SELECT empno,ename,job FROM emp

GRANT all on v1 to c##ravi;

SELECT * FROM c##batch4pm.v1;

empno ename job

CREATE VIEW v2

AS

SELECT e.ename,e.sal,d.dname,d.loc

FROM emp e, dept d

WHERE e.deptno=d.deptno;

SELECT * FROM v2;

Indexes:

- Index is Database Object.
- It is used to improve the performance of data retrieval.
- Index will be created on a column which is frequently used for data retrieval.

Syntax:

```
CREATE INDEX <index_name>
ON <table_name>(<column>);
```

ORACLE BOOK

INDEX

chapter name	page no
DDL commands	10
DML	25
DCL	40
Joins	60

set autotrace on explain

SELECT * FROM emp WHERE sal>3000; --table scan

100 records => 100 comparisons

CREATE INDEX i1 ON emp(sal);

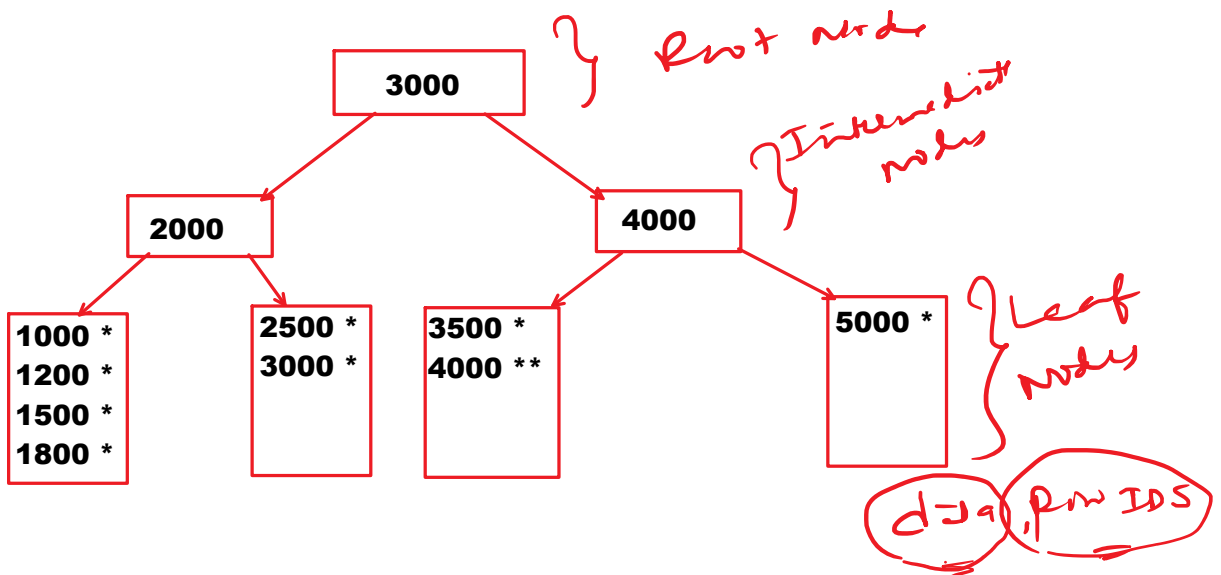
SELECT * FROM emp WHERE sal>3000; --index scan

as on avrg => maximum half of the comparisons

index scan is faster than table scan

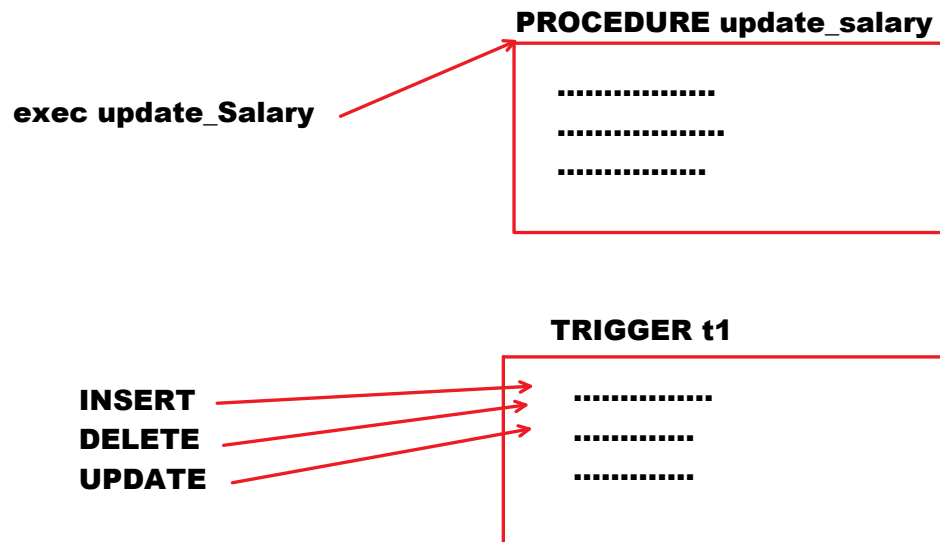
sal

4000
3500
5000
3000
4000
1500
1800
1200
2500
1000



Triggers:

- **Trigger is a named block of statements that gets executed automatically when we perform DML operation.**



- **Trigger can be used to control the DML operations**

Ex:

don't allow the user to update records on Sunday

- **can be used for auditing the tables or databases.**

Ex:

user	datetime	operation
------	----------	-----------

Syntax:

```
CREATE OR REPLACE TRIGGER <trigger_name>  
BEFORE / AFTER INSERT OR UPDATE OR DELETE  
ON <table_name>  
[FOR EACH ROW]  
DECLARE  
    --declare the variable  
BEGIN  
    --executable statement  
END;  
/
```

define a trigger not to allow the user to perform DML operations on Sunday:

```
CREATE OR REPLACE TRIGGER t1
BEFORE INSERT OR UPDATE OR DELETE
ON emp
BEGIN
  IF to_char(sysdate,'dy')='tue' THEN
    RAISE_APPLICATION_ERROR(-20050,'DMLs not
    allowed on Sunday');
  END IF;
END;
/
```

%rowtype:

- It is attribute related data type.
- It is used to hold entire row.
- decrease no of variables.

Display the emp record of given empno:

Enter value for empno: 7499

ALLEN 5000 500 23-dec-1981 manager 20

r emp%rowtype;

r

empno	ename	job	sal	comm	hiredate	deptno	mgr
7499	ALLEN	clerk	5000	500	..	..	..

Handwritten annotations:
Arrows point from the table cells to the following text:
r.ename (from ename)
r.job (from job)
r.sal (from sal)
r.hiredate (from hiredate)

Program to display emp record of given empno:

```
DECLARE
  v_empno emp.empno%type;
  r emp%rowtype;
BEGIN
  v_empno := &empno;

  SELECT * INTO r FROM emp
```

WHERE empno=v_empno;

**dbms_output.put_line(r.ename || ' ' || r.sal || ' ' ||
r.hiredate || ' ' || r.deptno);**

END;

/

SQL

PL/SQL